

ZigZag Trie: A Novel Index for Contextual Queries

Ling Li¹, Daniel Gibney², Sharma V. Thankachan³, Rahul Shah⁴, Grigorios Loukides¹, and Solon P. Pissis⁵

¹King’s College London, UK ²University of Texas at Dallas, USA

³North Carolina State University, USA ⁴Louisiana State University, USA ⁵The Cyprus Institute, Cyprus

Abstract—There is increasing interest in queries about the *context* of a string P in a longer text T , i.e., the set of all string pairs (L, R) , with $|L| = |R| = q$, for a given q , such that the string LPR occurs in T . Such queries are important in domains as diverse as bioinformatics, log analysis, and text analytics but are challenging to answer efficiently. This is because the length of T in applications is massive and existing indexes do not directly encode the context of a given P , which is key for answering retrieval queries efficiently. Our work introduces the **ZigZag Trie (ZZT)**, a new index to specifically address these challenges. This index reorganizes the text so that, for any P , all possible strings L and R growing symmetrically around P are grouped into a common subtree of the index, allowing their efficient retrieval. We show how to construct the **ZZT** of T , which has size $\mathcal{O}(n)$ where $n = |T|$, in $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n)$ space.

On top of **ZZT**, we design specialized indexes that, for a query pattern P of length m , answer four new types of contextual queries: (I) finding the longest string LPR that occurs at least τ times in T , for a fixed τ ; (II) finding the longest string LPR that occurs in at least τ texts of a text collection, for a fixed τ ; (III) reporting the total number of distinct contexts of P in T ; and (IV) retrieving the k pairs (L, R) of P with the highest scores according to a given scoring function. Our indexes answer queries of type I, II, and III in *optimal* time, and of type IV in *near-optimal* time. Moreover, their size, construction space, and construction time are linear or near-linear in n , given **ZZT**. Using real billion-letter datasets from different domains, we show that our indexes answer queries orders of magnitude faster than baselines, which are based on traditional text indexes or on the state of the art, and perform similarly or better in terms of index size, construction space, and construction time.

I. INTRODUCTION

Strings are ubiquitous in application domains such as genomics [1], natural language processing [2], log analysis [3], and user-behavior modeling [4]. At the heart of these applications, there is *text indexing* [5]. This problem asks for preprocessing a string T (text) into a compact data structure that supports efficient pattern matching, e.g., *report* the set of starting positions of a *pattern* P in T or *count* this set’s size.

There has been increasing interest (e.g., [6], [7], [8]) in *contextual queries*, whose focus is *not* only on the pattern P itself but on the string L and R which occurs immediately to the left and to the right of P in T , respectively. The *set* of such string pairs (L, R) with $|L| = |R| = q$ is referred to as the *context* of P in T [6] and denoted by $C_T(P, q)$; L and R is referred to as the *left* and *right* context of P in T , respectively. Motivated by bioinformatics and text or log analysis applications, Navarro [6] considered the problem of reporting $C_T(P, q)$, while Li et al. in their ICDE 2026 paper [8] considered the problem of counting the size of $C_T(P, q)$.

$Z_1 : ba$
 $Z_2 : anba$
 $Z_3 : naanba$
 $Z_4 : annaa$
 $Z_5 : naa$
 $Z_6 : a$

(a)

$Z_6 : a$
 $Z_2 : anba$
 $Z_4 : annaa$
 $Z_1 : ba$
 $Z_5 : naa$
 $Z_3 : naanba$

(b)

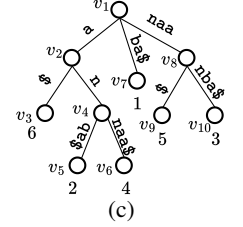


Fig. 1: (a) The ZigZag strings Z_i for $T = banana$. (b) The Z_i ’s sorted lexicographically. (c) **ZZT**(T), where the subscript i of each node v_i denotes its *preorder rank* and the special terminating symbol $\$$ marks the end of a ZigZag string.

A. The ZigZag Trie

We propose the **ZigZag Trie (ZZT)**, a novel index specifically designed for answering contextual queries. The key idea in the design of **ZZT** is to define, for each position $i \in [1, n]$ of T , a *ZigZag string* $Z_i = T[i]T[i+1]T[i-1]T[i+2]T[i-2] \dots$ that “walks” alternately to the right and to the left of T . These n strings are sorted lexicographically, and the **ZZT** of T , denoted by **ZZT**(T), is the compacted trie built over these strings.

Example 1. Let $T = banana$. The ZigZag strings constructed for all positions $i \in [1, 6]$ are shown in Fig. 1a. For instance, $Z_3 = T[3]T[4]T[2]T[5]T[1]T[6] = naanba$. The list of lexicographically sorted ZigZag strings and **ZZT**(T) are shown in Fig. 1b and 1c, respectively.

Why is the ZigZag Trie useful?

The utility of **ZZT**(T) stems from its construction based on ZigZag strings: for any query pattern P of length m , **ZZT**(T) *directly encodes* the contextual information of P in the subtree induced by P in **ZZT**(T). Namely, for any q , all (L, R) string pairs in the context $C_T(P, q)$ are encoded as descendants of the locus of the *transformed* P (i.e., the point in **ZZT**(T), where $ZZ(P) = P[\lceil m/2 \rceil]P[\lceil m/2 \rceil + 1]P[\lceil m/2 \rceil - 1] \dots$ ends). All of these descendants are at string depth $m + 2q$. This property is *key* for solving diverse contextual queries that impose constraints on this subtree, and it is *not* offered by existing indexes.

Example 2 (Cont’d from Example 1). For $P = a$, $C_T(P, 1) = \{(b, n), (n, n)\}$, and the locus of $ZZ(P) = a$ is the node v_2 in Fig. 1c. The strings Z_6 , Z_2 , and Z_4 , which start with a , lie in the subtree of v_2 ; the next letter n in the subtree corresponds to the two strings R , n and n , in $C_T(P, 1)$, and the outgoing edges of v_4 start with b and n , the two strings L in $C_T(P, 1)$.

B. Contextual Queries

We showcase the usefulness of ZZT by formalizing four types of contextual queries that arise in various applications and by presenting results for query answering on real datasets.

Longest Frequent Contextual Superstring (LFCS).

Problem 1 (LFCS). A text T of length n and an integer $\tau \in [1, n]$ are given for preprocessing. Given a pattern P of length m , report a longest string LPR , with $|L| = |R|$, that occurs at least τ times in T , if such a string exists.

An important application of LFCS comes from financial computing, as it summarizes the context around a stock trading event without making stronger claims about stock price prediction or performance. Here, T is a discretized time-series of the price of a stock over time and P is a salient trading event, such as a price spike. The LFCS output then corresponds to pre- and post-event price behavior associated with the event that is both persistent and recurring, and hence economically meaningful [9]. Thus, LFCS outputs can help in designing algorithmic trading strategies or detecting anomalies (e.g., when in future data the outputs appear with a much different frequency) [9]. Another application of LFCS is in log analysis. Here, T is a sequence of system events, P models a key system event (e.g., “login”), and L and R model events that precede and succeed the key event. LFCS can help construct a *structured event template* [10] by finding the longest sequence of events LPR (i.e., events in a window around P) with $|L| = |R|$ which occurs frequently. Such sequences capture typical behavior and are used to detect deviations or system failures in future data.

Example 3. We constructed a data structure for answering LFCS queries on the RUET OJ log dataset [11] with $\tau = 20$ and asked 3 queries, each modeling a different key system event. Table I shows these queries, their answers, and the typical behavior they indeed capture.

TABLE I: LFCS on a log dataset.

P	Answer	Typical behavior
auth	login->auth->home	Successful authentication flow
signup	login->signup->action	Successful signup flow
compile	submit_page->compile->submission	Page submission pipeline

Longest Common Contextual Superstring (LCCS).

Problem 2 (LCCS). A collection $\mathcal{C} = \{T_1, T_2, \dots, T_c\}$ of $c > 1$ texts of total length N and an integer $\tau \in [1, c]$ are given for preprocessing. Given a pattern P of length m , report a longest string LPR , with $|L| = |R|$, that occurs in at least τ texts of $\mathcal{C}$, if such a string exists.

LCCS is also motivated by applications in financial computing and log analysis. In financial computing, the setting is similar to that of LFCS but now T_i ’s model prices of different stocks and LPR models common price behavior (e.g., LPR can be a *following motif* occurring when the price of one stock in a window imitates the price behavior of another [12]). In log analysis, the setting is again similar to that of LFCS but

now T_i ’s are logs of different servers and LPR must occur in sufficiently many server logs to be deemed typical [13].

Example 4. We constructed a data structure for answering LCCS queries on the same dataset as in Example 3 but now using $\tau = 5$ and treating the sequence for each of the $c = 5$ IP addresses as a separate text T_i . Table II shows that the answers of the 3 queries for key system events we asked indeed capture typical behavior (performed by all 5 IP addresses).

TABLE II: LCCS on a log dataset.

P	Answer	Typical behavior
auth	home->login->auth-> home->archive	Authentication moves from home, to login, then to home, and next to browsing the archive content
signup	home->login->signup-> action->login	Signup after visiting home and login, followed by an initial account action and re-login
submit_page	problem->problem_detail-> submit_page->compile-> submission	Users view a problem then its details, and then go to the submit page to compile and submit it

Contextual Complexity (CC).

Problem 3 (CC). A text T of length n is given for preprocessing. Given a pattern P of length m , report the contextual complexity $CC(P, T) = \sum_q |\mathcal{C}_T(P, q)|$ of P in T .

The CC problem measures the number of distinct context pairs of P of any length $q = |L| = |R|$ (i.e., how “rich” is the context set of P). It is motivated by applications in text analytics and biology. In text analytics, it can help compare the *polysemy numbers* [14] of words (i.e., the number of different related meanings) [8], [15]. Consider, for example, two words (patterns P_1, P_2) that have similar frequencies $\text{occ}_T(P_1) \approx \text{occ}_T(P_2)$ in a text T but for which $CC(P_1, T) > CC(P_2, T)$. Then, P_1 is in between more different string pairs (L, R) which may give more different meanings to it. As a more frequent P has intuitively a greater chance to have a higher $CC(P, T)$, one can instead use the ratio $CC(P, T) / \text{occ}_T(P)$. As another application, in biology, a DNA motif P with large $CC(P, T)$ occurs in between many distinct *flanking-sequence* [16] environments. This indicates that the motif is reused across multiple cis-regulatory settings and its functional effect may be modulated by local sequence context, including nearby transcription-factor binding sites [16].

Example 5. We constructed a data structure for answering CC queries on the text from the book “On the Origin of Species” [17]. In Table III, *place* and *origin* have similar frequencies but different contextual complexities. Indeed, *place* has also a larger polysemy number, as measured by the number of WordNet synsets [18]. The word *case* has the largest frequency and also the largest contextual complexity, but its ratio is in between those of the other two words. Indeed, its polysemy number is also in between theirs.

Top- k Contextual Pattern Retrieval (TCPR). Notably, we lift the Contextual Pattern Matching problem, introduced by Navarro [6], to its more general, top- k counterpart. To this end, we employ a *scoring function* $f_T : \Sigma^* \rightarrow \mathbb{R}_{\geq 0}$ that gets as input a substring of T and outputs a nonnegative real number.

TABLE III: Polysemy numbers comparison based on CC.

P	$\text{occ}_T(P)$	$\text{CC}(P, T)$	$\frac{\text{CC}(P, T)}{\text{occ}_T(P)}$	Polysemy Number
place	57	2691932	47226.9	32
origin	52	1117453	21489.5	6
case	282	12571831	44581.0	22

We will assume oracle access to f_T (i.e., we can read its values in $\mathcal{O}(1)$ time) after an $\mathcal{O}(n)$ -time preprocessing of T .

Problem 4 (TCPR). A text T of length n and a scoring function f_T are given for preprocessing. Given a pattern P of length m and integers $q \geq 0$ and $k > 0$, report k string pairs (L, R) from $\mathcal{C}_T(P, q)$ with the highest $f_T(LPR)$ scores, if such pairs exist.

Unlike LFCS and LCCS which have a single (L, R) pair as part of their output, TCPR outputs the k pairs (L, R) with the highest f_T scores. The function f_T captures the importance of pairs from $\mathcal{C}_T(P, q)$ using well-known notions such as their *frequency* or *span* [19], [20] in T . A prime application of TCPR comes from bioinformatics. Here, P corresponds to a specific pattern of interest (e.g., a CpG site in a certain type of DNA region called CpG island [21], which helps to study the evolutionary history of mammalian genomes [21]) and the pairs (L, R) correspond to *flanking sequences*, which are highly relevant for understanding the role of genes [22] and DNA shape fluctuations [23]. A TCPR query returns the k most highly ranked flanking sequences, based on their frequency which is used in the study of [21]. Another application comes from text analysis, where P corresponds to a word or phrase of interest and the top- k pairs (L, R) to the words or phrases that provide different, highly-important meanings to P [15], [8]. For example, when $P = \text{bank}$ the top-2 pairs (L, R) when $q = 1$ could be $(\text{south}, \text{Thames})$ and $(\text{Savings}, \text{Ukraine})$. In the first case, P corresponds to a river bank, while in the second to a financial institution.

Example 6. We constructed a TCPR data structure on the same text T as in Example 5 with f_T outputting the frequency of strings LPR in T . For $P = \text{nature}$, $q = 4$, and $k = 20$, Table IV shows two representative contexts that provide entirely different meanings to the word *nature*.

TABLE IV: TCPR on a book dataset.

P	Answer	Meaning
nature	animals and plants throughout nature struggle for life most	the natural world
nature	it is in human nature to value any novelty	the inherent character of a person

C. Contributions

1. Our central contribution is the ZigZag Trie. For a string T of length n , $\text{ZZT}(T)$ occupies $\mathcal{O}(n)$ space and locates all $\text{occ}_T(P)$ occurrences of a pattern P of length m in T in the optimal $\mathcal{O}(m + \text{occ}_T(P))$ time. We present an algorithm to construct $\text{ZZT}(T)$ in $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n)$ space. Crucially, we show how the ZZT can be used to construct specialized indexes that answer LFCS, LCCS, and CC queries in *optimal* time, and TCPR queries in *near-optimal* time.

2. We propose an algorithm that, given $\text{ZZT}(T)$, constructs an $\mathcal{O}(n)$ -size index that answers LFCS queries in the optimal $\mathcal{O}(m)$ time. This index can be constructed in $\mathcal{O}(n)$ time and space given $\text{ZZT}(T)$. To arrive at this result, we rely on the structural properties of $\text{ZZT}(T)$.

3. We propose an algorithm that, given $\text{ZZT}(T)$, constructs an $\mathcal{O}(N)$ -size index that answers LCCS queries in the optimal $\mathcal{O}(m)$ time. This index can be constructed in $\mathcal{O}(N)$ time and space, given $\text{ZZT}(T)$. To arrive at this result, we further utilize an algorithm for the *color set size* problem [24].

4. We propose an algorithm that, given $\text{ZZT}(T)$, constructs an $\mathcal{O}(n)$ -size index that answers CC queries in the optimal $\mathcal{O}(m)$ time. This index can be constructed in $\mathcal{O}(n)$ time and space given $\text{ZZT}(T)$. The main idea in the construction is to count, from every possible locus of a query pattern of length m , and every q , the number of distinct contextual extensions at string depth $m + 2q$, performed using dynamic programming.

5. We propose an algorithm that, given $\text{ZZT}(T)$, constructs an $\tilde{\mathcal{O}}(n)$ -size index that answers TCPR queries in near-optimal $\mathcal{O}(m) + \tilde{\mathcal{O}}(k)$ time. This index can be constructed in $\tilde{\mathcal{O}}(n)$ time and space, using a data structure for top- k orthogonal range reporting [25]. We also propose a parameterized version of our index that outperforms the non-parameterized one in all efficiency measures [8]: query time, index size, construction space, and construction time. Both versions support a wide variety of f_T functions from the literature [19], [20], [26].

6. We compare our specialized indexes to baseline indexes, as there are no *out-of-the-box* solutions for our problems. The baselines for LFCS, LCCS, and CC are based on the suffix tree [27], the textbook index for answering pattern matching queries in optimal time. The baseline for TCPR uses the state-of-the-art index for reporting $\mathcal{C}_T(P, q)$ by Navarro [6]. Unlike our indexes, these baselines have query times that depend on the number $\text{occ}_T(P)$ of occurrences of P in T or on the size of $\mathcal{C}_T(P, q)$, which are in the order of thousands in practice [6], [8]. Indeed, using real billion-letter datasets from different domains, we show that our indexes outperform the baselines by *orders of magnitude* in terms of query time and perform similarly or better in terms of index size, construction space, and construction time. Thus, they offer a *desirable trade-off*: answering queries much more efficiently with a similar size and one-off cost in terms of time and space to be constructed.

D. Organization

Section II discusses some basic concepts and Section III our ZZT index. Sections IV, V, VI, and VII discuss the baseline and our indexes for LFCS, LCCS, CC, and TCPR. Section VIII discusses related work, Section IX presents our experimental evaluation, and Section X concludes the paper.

II. PRELIMINARIES

Strings. An *alphabet* Σ is a finite set of elements called *letters*. We consider throughout that Σ is an integer alphabet. For a string $T = T[1..n] \in \Sigma^n$, we denote its length n by $|T|$ and its i th letter by $T[i]$. A *substring* of T starting at position i and

ending at position j of T is denoted by $T[i..j]$. A substring S of T may have multiple occurrences in T . We thus characterize an *occurrence* of S in T by its *starting position* $i \in [1, n]$; i.e., $S = T[i..i + |S| - 1]$. A substring of the form $T[i..n]$ is a *suffix* of T . A substring of the form $T[1..i]$ is a *prefix* of T . The *reverse* of T is denoted by T^R . For example, for string $T = \text{CTAAG}$, the reverse is $T^R = \text{GAATC}$. The *concatenation* of strings X, Y is denoted by $X \cdot Y$ (or by XY).

Compacted Tries, Suffix Trees, Suffix and LCP Arrays. Let $\mathcal{X}$ be a set of strings. The *compacted trie* of $\mathcal{X}$ is the trie of $\mathcal{X}$ in which each maximal branchless path is replaced by a single edge whose label is the concatenation of its edge labels. We append a terminating symbol $\$$ to every string, such that $\$$ occurs only at the last position of each string and is the lexicographically smallest. For a node v in a compacted trie $\mathcal{T}$, $\text{str}(v)$ is the concatenation of edge labels on the root-to- v path. We define the *string depth* of a node v as $\text{sd}(v) = |\text{str}(v)|$. For example, consider v_{10} in Fig. 1c. Its edge with label $\text{nba}\$$ replaces a path of edges $\text{n}, \text{b}, \text{a}$, $\text{str}(v_{10}) = \text{naanba}\$$, and $\text{sd}(v_{10}) = 7$. The *locus* of a string P in $\mathcal{T}$ is the pair $(v, |P|)$, where v is the node with the smallest $\text{sd}(v)$ such that P is a prefix of $\text{str}(v)$. When $\text{str}(v) = P$, $(v, |P|)$ corresponds to an *explicit* node; otherwise, it corresponds to an *implicit* node. For example, the locus of $P = \text{na}$ in Fig. 1c is $(v_8, 2)$, since v_8 has the smallest sd such that P is a prefix of $\text{str}(v_8)$. We denote the *parent* of a node v by $\text{parent}(v)$.

The *suffix tree* of a string $T[1..n]$, denoted by $\text{ST}(T)$, is the compacted trie of the set of suffixes of T with its leaves, $l_1, \dots, l_n$, ordered in ascending lexicographical rank of the corresponding suffixes [27]; see Fig. 2 for an example. We associate each leaf with an id, $1, \dots, n$, so that the i th leaf from left to right has id equal to i . The suffix tree can be constructed in $\mathcal{O}(n)$ time for integer alphabets of polynomial size in n [28]. The suffix tree of a collection $\mathcal{C}$ of strings of total length N can be similarly constructed in $\mathcal{O}(N)$ time. We denote it by $\text{ST}(\mathcal{C})$.

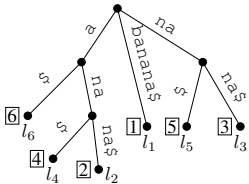


Fig. 2: $\text{ST}(T)$ for $T = \text{banana}$; each leaf id is in a square.

$\text{ISA}[1..6] = [4, 3, 6, 2, 5, 1]$.

We use $\text{LCE}_T(i, j)$ to denote the length of the *longest common prefix* (LCP) of $T[i..n]$ and $T[j..n]$. For example, $\text{LCE}_T(3, 5) = 2$ in Fig. 2 because the LCP of $T[3..6] = \text{nana}$ and $T[5..6] = \text{na}$ is na , which has length 2. Note that $\text{LCE}_T(i, j)$ can be computed in $\mathcal{O}(1)$ time after an $\mathcal{O}(n)$ -time preprocessing of the suffix tree [30]. We analogously use $\text{LCE}_T(i, j)$ to denote the length of the *longest common suffix* (LCS) of $T[1..i]$ and $T[1..j]$. Similarly, this quantity can be

computed in $\mathcal{O}(1)$ time after an $\mathcal{O}(n)$ -time preprocessing.

The $\text{LCP}[1..n]$ array of a string $T[1..n]$ stores the length of the LCP of lexicographically adjacent suffixes [29]. For $j > 1$, $\text{LCP}[j]$ stores the length of the LCP between the suffixes starting at $\text{SA}[j-1]$ and $\text{SA}[j]$, and $\text{LCP}[1] = 0$. For example, for $T = \text{banana}$, $\text{LCP}[1..6] = [0, 1, 3, 0, 0, 2]$. Given SA , the LCP array of T can be computed in $\mathcal{O}(n)$ time [31].

Top- k Queries for Orthogonal Range Reporting. Let S be a set of n weighted points in $\mathbb{R}^d$, where each point $p \in S$ has a real-valued weight $w(p)$. Preprocess S into a data structure so that given: (1) an orthogonal query box $Q = \prod_{i=1}^d [a_i, b_i]$; and (2) an integer $k \in [1, n]$, the data structure can efficiently report the k points in $S \cap Q$ with the highest weights. Our data structure for TCPR is built upon the one from [25], which uses $\mathcal{O}(n \log^d n)$ space with $\mathcal{O}(\log^d n + k)$ query time for arbitrary order reporting, where $d = \mathcal{O}(1)$.

III. THE ZIGZAG TRIE

Definitions. Let $T[1..n]$ be a string of length n over alphabet Σ . For every position $i \in [1, n]$, we define the *ZigZag* string $Z_i(T)$ as the sequence $Z_i = T[i] T[i+1] T[i-1] T[i+2] T[i-2] \dots$, where the construction terminates as soon as the next index falls outside of $[1, n]$; we drop (T) from $Z_i(T)$ when the context is clear. More formally, for all $i \in [1, n]$, we have

$$|Z_i| = \max \left\{ k \geq 1 : i + (-1)^k \left\lfloor \frac{k}{2} \right\rfloor \in [1, n] \right\},$$

and then, for all $k \in [1, |Z_i|]$, we have

$$Z_i[k] = T \left[i + (-1)^k \left\lfloor \frac{k}{2} \right\rfloor \right].$$

The above equations tell us that given *any* i, k , we can determine whether $Z_i[k]$ is defined and, if so, access it in $\mathcal{O}(1)$ time. We will thus assume oracle access to $\{Z_1, Z_2, \dots, Z_n\}$. From the above, it is also easy to see that the prefix $Z_i[1..\ell]$ of length ℓ of Z_i occurs at position $(i - \lceil \ell/2 \rceil + 1)$ of T .

The *ZigZag Trie* (ZZT) of T , denoted by $\text{ZZT}(T)$, is the compacted trie over all n ZigZag strings $\{Z_1, Z_2, \dots, Z_n\}$; we append a terminating symbol $\$ \notin \Sigma$, that is lexicographically smaller than every letter in Σ , to each Z_i . Leaf and internal nodes of $\text{ZZT}(T)$ are defined analogously to those of $\text{ST}(T)$.

We further define: (I) the *ZigZag Array* (ZZA) of T , denoted by $\text{ZZA}(T)$: $\text{ZZA}(T)[i] = j$ if Z_j is the i th ZigZag string in lexicographic order, and (II) the *ZigZag LCP* array of T , denoted by $\text{ZZ-LCP}(T)$: $\text{ZZ-LCP}(T)[1] = 0$, and $\text{ZZ-LCP}(T)[j]$, for $j > 1$, stores the length of the longest common prefix of $Z_{\text{ZZA}(T)[j-1]}$ and $Z_{\text{ZZA}(T)[j]}$. We use $\text{ZZA}(T)$ and $\text{ZZ-LCP}(T)$ to efficiently construct $\text{ZZT}(T)$.

Example 7. For $T = \text{banana}$, we have the following.

i	Z_i	$\text{ZZA}(T)[i]$	$Z_{\text{ZZA}(T)[i]}$	$\text{ZZ-LCP}(T)[i]$
1	ba	6	a	0
2	anba	2	anba	1
3	naanba	4	annaa	2
4	annaa	1	ba	0
5	naa	5	naa	0
6	a	3	naanba	3

Construction. To construct $\text{ZZT}(T)$, we need to sort the ZigZag strings $\{Z_1, Z_2, \dots, Z_n\}$ lexicographically. However, explicitly constructing and then sorting them is prohibitively expensive, as it takes $\mathcal{O}(\sum_{i \in [1, n]} |Z_i|) = \mathcal{O}(n^2)$ time. In response, we show how they can be sorted in $\mathcal{O}(n \log n)$ time *without being explicitly constructed*. This leads to an $\mathcal{O}(n \log n)$ -time algorithm for constructing $\text{ZZT}(T)$.

First, construct $\text{ST}(T)$ and $\text{ST}(T^R)$ to support $\mathcal{O}(1)$ -time LCE queries in both directions. To compare any two ZigZag strings Z_i and Z_j lexicographically, we compute:

- $r := \overleftarrow{\text{LCE}}_T(i, j)$, i.e., the length of the longest common suffix of prefixes $T[1..i]$ and $T[1..j]$;
- $f := \text{LCE}_T(i+1, j+1)$, i.e., the length of the longest common prefix of suffixes $T[i+1..n]$ and $T[j+1..n]$.

If $r \leq f$, then the two strings agree through the first $2r$ positions, and the first mismatch occurs at position $2r+1$; otherwise, they agree through position $2f+1$, and the first mismatch occurs at position $2f+2$. Hence, they can be compared in $\mathcal{O}(1)$ time. This enables sorting the strings in $\{Z_1, Z_2, \dots, Z_n\}$ without explicitly constructing them, in $\mathcal{O}(n \log n)$ time, using any optimal comparison-based sorting algorithm, thereby producing $\text{ZZA}(T)$. Then, for every pair of successive entries of $\text{ZZA}(T)$, we use two LCE queries to infer the LCP value yielding $\text{ZZ-LCP}(T)$. Finally, we use the algorithm by Kasai et al. [31] to construct $\text{ZZT}(T)$ from $\text{ZZA}(T)$ and $\text{ZZ-LCP}(T)$ in $\mathcal{O}(n)$ time, analogously to how a suffix tree can be built from a suffix array and LCP array.

Example 8. Consider the comparison of $Z_2 = \text{anba}$ and $Z_4 = \text{annaa}$ for $T = \text{banana}$. We compute $r := \overleftarrow{\text{LCE}}_T(2, 4) = 1$ and $f := \text{LCE}_T(3, 5) = 2$. Since $r \leq f$, the first mismatch occurs at position $2r+1 = 3$: $Z_2[3] = b \neq Z_4[3] = n$. As $b < n$, we infer $Z_2 < Z_4$ in $\mathcal{O}(1)$ time.

Each branching node of $\text{ZZT}(T)$ is augmented with a dictionary that supports $\mathcal{O}(1)$ -time access to the edges based on the first letter (key) of their label. The total size of the dictionaries is $\mathcal{O}(n)$, and they can be constructed in $\mathcal{O}(n \log n)$ time [32]. We have arrived at the following result.

Theorem 1. For every text T of length n , $\text{ZZT}(T)$ can be constructed in $\mathcal{O}(n \log n)$ time using $\mathcal{O}(n)$ space. The data structure occupies $\mathcal{O}(n)$ space.

Querying. To search for a pattern P of length m in $\text{ZZT}(T)$, we first transform P into its ZigZag counterpart. Let $x := \lceil m/2 \rceil$. The ZigZag transformation $\text{ZZ}(P)$ is defined as:

$$\text{ZZ}(P) = P[x]P[x+1]P[x-1]P[x+2]P[x-2] \dots$$

with $|\text{ZZ}(P)| = m$. More formally, for all $k \in [1, m]$,

$$\text{ZZ}(P)[k] = P \left[x + (-1)^k \cdot \left\lfloor \frac{k}{2} \right\rfloor \right].$$

Example 9. $\text{ZZ}(abcd) = bcad$ and $\text{ZZ}(abcde) = cdbea$.

This transformation is clearly implementable in $\mathcal{O}(m)$ time. Then, finding the locus of $\text{ZZ}(P)$ in $\text{ZZT}(T)$ takes $\mathcal{O}(m)$ time. Thus, like with $\text{ST}(T)$ [27], we can locate all $\text{occ}_T(P)$

occurrences of P in T in the optimal $\mathcal{O}(m + \text{occ}_T(P))$ time using $\text{ZZT}(T)$. Note that it is also straightforward to implement the inverse of ZZ in $\mathcal{O}(m)$ time: for all $i \in [1, m]$,

$$P[i] = \begin{cases} \text{ZZ}(P)[2x - 2i + 1] & \text{if } i \leq x, \\ \text{ZZ}(P)[2(i - x)] & \text{if } i > x. \end{cases}$$

Similarly to suffix trees, we can trivially generalize the ZZT for a collection $\mathcal{C} = \{T_1, \dots, T_c\}$ of c strings of total length N , which we denote by $\text{ZZT}(\mathcal{C})$. Yet, there are two differences: (I) The compacted trie is now constructed over the *union* of the ZigZag strings of every string in $\mathcal{C}$. (II) We append a unique terminating symbol $\$i \notin \Sigma$ to the ZigZag strings of T_i , for all $i \in [1, c]$. The construction clearly takes $\mathcal{O}(N \log N)$ time.

IV. INDEXES FOR LFCS

A. Baseline Index for LFCS

Construction. Construct the suffix trees $\text{ST}(T)$ and $\text{ST}(T^R)$ for the text T and its reverse T^R . Using a DFS on $\text{ST}(T)$, for each node v , add a pointer to a descendant u of v with maximum string depth such that the number of leaves $L(u)$ in the subtree rooted at u is at least τ ; such a pointer may not exist, and u may be v itself. This pointer will be used to get the *longest* frequent (i.e., occurring at least τ times) extension LPR of P , for a query P . We then perform the same preprocessing over $\text{ST}(T^R)$. The key property is Lemma 1.

Lemma 1. Let LPR be an optimal solution to LFCS. Consider the locus of PR and $(LP)^R$ in $\text{ST}(T)$ and $\text{ST}(T^R)$, respectively. At least one of these loci is explicit.

Proof. Assume towards contradiction that both loci are implicit. Since the locus of PR in $\text{ST}(T)$ is implicit, it lies inside an edge label and there exists a *unique* letter a such that PRa is still represented on that same edge; therefore PRa occurs in T . Symmetrically, since the locus of $(LP)^R$ in $\text{ST}(T^R)$ is implicit, there exists a *unique* letter b such that bLP occurs in T . Thus, every occurrence of LPR in T is preceded by b and followed by a and so $\text{occ}_T(bLPRa) = \text{occ}_T(LPR) \geq \tau$. Moreover, since $|L| = |R|$, $|L| + 1 = |R| + 1$. Thus, $bLPRa$ is a feasible solution to LFCS. Since it is longer than LPR , this contradicts the optimality of LPR . $\square$

Querying. We first locate the locus $(v, |P|)$ of the query pattern P in $\text{ST}(T)$ and perform a DFS on the subtree rooted at v , collecting every node u with $L(u) \geq \tau$. Each such node u defines a possible frequent right extension PR of P .

For each PR , we locate the locus $(w, |PR|)$ of $(PR)^R$ in $\text{ST}(T^R)$, and use the pointer of w to obtain the longest frequent left extension of PR . We truncate L if $|L| > |R|$ so that both have the same length $|R|$, as required. (If $|L| < |R|$, then no candidate is constructed for node u .) For each collected node u , we form a candidate LPR . Every such candidate satisfies the frequency threshold and the condition $|L| = |R|$ required by Problem 1. We output a *longest* candidate to also meet the optimization goal.

The query algorithm described above is performed again starting from the locus of P^R in $ST(T^R)$ and searching in $ST(T)$. The correctness of this algorithm follows by Lemma 1.

Complexity. The index has $\mathcal{O}(n)$ size as $ST(T)$ and $ST(T^R)$ have $\mathcal{O}(n)$ size. The construction time and space is $\mathcal{O}(n)$, because constructing and preprocessing $ST(T)$ and $ST(T^R)$ via DFS all take $\mathcal{O}(n)$ time and use $\mathcal{O}(n)$ space. At query time, locating $(v, |P|)$ takes $\mathcal{O}(m)$ time, and collecting all candidates PR takes $\mathcal{O}(\text{occ}_T(P))$ time. For each PR , we recover the corresponding left context by traversing upward from a leaf in the subtree of w , which takes $\mathcal{O}(\text{occ}_T(P))$ time. Therefore, the overall query time is $\mathcal{O}(m + \text{occ}_T(P)^2)$.

B. Our Index for LFCS

Our index avoids the expensive “coordinated” traversal of the suffix trees of the baseline, by exploiting the structure of ZZT. It adds pointers to $ZZT(T)$ that *directly* lead to nodes from which we get query answers. This yields the optimal $\mathcal{O}(m)$ query time, which does *not* depend on $\text{occ}_T(P)$.

Construction. Construct $ZZT(T)$ using Theorem 1. Then perform a DFS and compute $L(v)$, for every node v in $ZZT(T)$. In a second DFS, for each node v , add a pointer to a descendant u of v with maximum string depth such that $L(u) \geq \tau$; such a pointer may not exist, and u may be v itself. This takes $\mathcal{O}(n)$ time. This pointer will be used to get the encoded longest possible frequent LPR .

Querying. Given a pattern P of length m , compute $ZZ(P)$ and locate its locus (v, m) in $ZZT(T)$ in $\mathcal{O}(m)$ time. If v has no stored pointer, then no such LPR string exists. Otherwise, follow the pointer from v to a node u : u encodes the *longest* frequent LPR as $ZZ(LPR) = \text{str}(u)$. If $|L| + |R|$ is not even, we truncate the last letter to guarantee that $|L| = |R|$. Let i be such that $\text{str}(u)$ is a prefix of string Z_i . Then LPR occurs at position $(i - \lceil \ell/2 \rceil + 1)$ of T , with $\ell = |LPR|$. The total query time is $\mathcal{O}(m)$. This is correct by how $ZZT(T)$ is constructed and by how the node pointers are constructed.

Example 10. Let $T = \text{banana}$, $\tau = 2$, and $P = n$. We compute $ZZ(P) = n$ and locate its locus $(v_8, 1)$ in $ZZT(T)$ (see Fig. 3). Following the pointer stored at v_8 , we arrive at v_8 itself, the deepest descendant u with $L(u) \geq 2$, which corresponds to the ZigZag prefix naa of length $3 = m + 2q$ with $q = 1$. It gives $L = a$, $R = a$, and $LPR = ana$, which occurs at positions 2 and 4 in T with frequency $2 \geq \tau$.

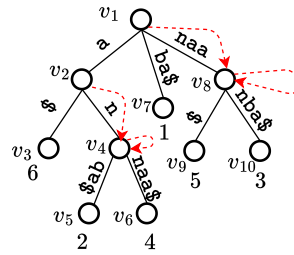


Fig. 3: $ZZT(T)$ of $T = \text{banana}$ with $\tau = 2$, annotated with pointers (dashed red arrows).

Main Result. From the above, we directly obtain:

Theorem 2. LFCS can be solved using an index of $\mathcal{O}(n)$ size supporting $\mathcal{O}(m)$ -time queries. The data structure can be constructed in $\mathcal{O}(n)$ time and space given $ZZT(T)$.

V. INDEXES FOR LCCS

A. Baseline Index for LCCS

This index is conceptually similar to the baseline index for LFCS, but now the forward suffix tree indexes the collection $\mathcal{C}$ and the reverse suffix tree indexes the reverse of each string in $\mathcal{C}$. Also, here, instead of leaf counts, we employ a coloring scheme in which *color counts* measure the number of distinct texts from $\mathcal{C}$ in which a candidate answer occurs.

Construction. Construct $ST(\mathcal{C})$ and $ST(\mathcal{C}^R)$, where $\mathcal{C}^R$ is $\{T_1^R, T_2^R, \dots, T_c^R\}$. Using a DFS on $ST(\mathcal{C})$, for each leaf ℓ originating from T_j , assign the color j to ℓ . For every node v in $ST(\mathcal{C})$, compute $C(v)$, the number of distinct colors in the subtree rooted at v using the linear-time algorithm in [24]. Using another DFS, for each node v , add a pointer to a descendant u of v with maximum string depth such that $C(u) \geq \tau$; such a pointer may not exist, and u may be v itself. We then perform the same preprocessing over $ST(\mathcal{C}^R)$.

Querying. The query algorithm is essentially the same as the one for the LFCS baseline; the only difference is that we use $C(v)$ instead of $L(v)$ to construct the PR candidates.

Complexity. The bounds are analogous to those of the LFCS baseline, but have the total string length N instead of n . Thus, the index has $\mathcal{O}(N)$ size, construction time, and construction space. The query time is $\mathcal{O}(m + \text{occ}_T(P)^2)$, as querying is the same as for LFCS: locating $(v, |P|)$ takes $\mathcal{O}(m)$ time, and the subsequent tree traversals cost $\mathcal{O}(\text{occ}_T(P)^2)$ in total.

B. Our Index for LCCS

Our index adds pointers to ZZT that *directly* lead to nodes from which we can get LCCS query answers in the optimal $\mathcal{O}(m)$ time, i.e., avoiding the term $\text{occ}_T(P)^2$ of the baseline.

Construction. Construct $ZZT(\mathcal{C})$. Using a DFS on $ZZT(\mathcal{C})$, for each leaf ℓ originating from T_j , assign the color j to ℓ . For every node v in $ZZT(\mathcal{C})$, compute $C(v)$, the number of distinct colors in the subtree rooted at v using the linear-time algorithm in [24]. Using another DFS, for each node v , add a pointer to a descendant u of v with maximum string depth such that $C(u) \geq \tau$; such a pointer may not exist, and u may be v itself. Given $ZZT(\mathcal{C})$, the construction takes $\mathcal{O}(N)$ time.

Querying. Given a pattern P of length m , compute $ZZ(P)$ and locate its locus (v, m) in $ZZT(\mathcal{C})$ in $\mathcal{O}(m)$ time. If v has no stored pointer, then no such LPR string exists. Otherwise, follow the pointer from v to a node u , and report node u that encodes the longest common LPR as $ZZ(LPR) = \text{str}(u)$. Note that we can locate a witness, i.e., a position where LPR occurs in a text in $\mathcal{C}$, similar to the LFCS query algorithm. The total query time is $\mathcal{O}(m)$. This is correct by how $ZZT(\mathcal{C})$ is constructed and by how the node pointers are constructed.

Main Result. From the above, we directly obtain:

Theorem 3. LCCS can be solved using an index of $\mathcal{O}(N)$ size supporting $\mathcal{O}(m)$ -time queries. The data structure can be constructed in $\mathcal{O}(N)$ time and space given $ZZT(\mathcal{C})$.

VI. INDEXES FOR CC

A. Baseline Index for CC

Construction. Construct $ST(T)$ and $ST(T^R)$.

Querying. We locate the locus $(v, |P|)$ of P in $ST(T)$, and then locate, for increasing $q \geq 0$, all the right extensions PR of P , with $|R| = q$, in the subtree of v . For each PR , we locate the locus of $(PR)^R$ in $ST(T^R)$, and count the left extensions LPR of PR , with $|L| = q$, in its subtree. Summing the counts over all PR and q yields the correct answer, as we exhaustively consider all valid loci pairs in $ST(T)$ and $ST(T^R)$.

Complexity. The construction takes $\mathcal{O}(n)$ time and space. The index size is $\mathcal{O}(n)$. The query time is $\mathcal{O}(m + |\mathcal{L}(v)|^2)$, where $\mathcal{L}(v)$ is the set of loci in the subtree rooted at v in $ST(T)$.

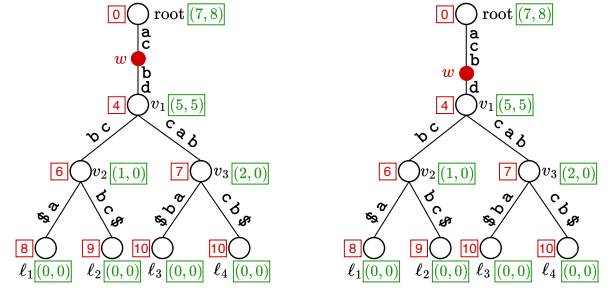
B. Our Index for CC

Our index exploits $ZZT(T)$ and auxiliary information computed via dynamic programming to compute $CC(P, T)$ in the optimal $\mathcal{O}(m)$ time. This is unlike the baseline index, whose query time has a large $|\mathcal{L}(v)|^2$ term, where v is the locus of P , due to the “coordinated” exploration of the two suffix trees.

Construction. Construct $ZZT(T)$ using Theorem 1. We compute $CC(P, T)$ via the number of contextual extensions LPR of P occurring in T over all $q = |L| = |R|$. Each extension corresponds to a path in the subtree rooted at the locus of $ZZ(P)$ in $ZZT(T)$. Since $|L| = |R| = q$, every extension has length $m + 2q$, where $m = |P|$. Thus, from the locus of $ZZ(P)$ at string depth m , we need to count the substrings in its subtree at string depths $m + 2q$, for all integers $q \geq 0$. We do this via dynamic programming.

To provide some intuition, let us denote a potential query string by P . Let $(v, |P|)$ denote the locus of $ZZ(P)$. The distance $d = \text{sd}(v) - |P|$ may be even or odd. If d is even, no single letter remains before reaching v , and the number of additional cross-edge length-2 fragments contributed by the outgoing edges of v is zero. For example, for $P = ac$ in Fig. 4a, the locus of $ZZ(P) = ac$ is $(v_1, 2)$ (i.e., node w). As $\text{sd}(v_1) = 4$ and $|P| = 2$, $d = \text{sd}(v_1) - |P| = 2$ is even, and no single letter remains until reaching v_1 . If d is odd, after traversing $\lfloor d/2 \rfloor$ length-2 fragments along the edge from locus $(v, |P|)$ to $(v, \text{sd}(v))$, one letter remains before reaching v . This letter can be combined with the first letter of every outgoing edge of v , contributing $|\text{ch}(v)|$ additional cross-edge length-2 fragments, where $\text{ch}(v)$ denotes the set of children of v . For example, for $P = bac$ in Fig. 4b, the locus of $ZZ(P) = acb$ is $(v_1, 3)$ (i.e., node w), and $d = \text{sd}(v_1) - |P| = 4 - 3 = 1$, which is odd. After traversing $\lfloor 1/2 \rfloor = 0$ length-2 fragments along the edge from $(v_1, 3)$ to $(v_1, 4)$, one letter, namely d , remains before reaching v_1 . This is combined with the first letter of each of the $|\text{ch}(v_1)| = 2$ outgoing edges of v_1 , contributing 2 additional cross-edge length-2 fragments.

We now formalize the above intuition. For an internal node v in $ZZT(T)$, let $u_1, u_2, \dots, u_k$ be its children, and let $d_i = \text{sd}(u_i) - \text{sd}(v)$ be the length of the edge label from v to u_i . To avoid counting the terminal symbol $\$$, we adjust this to



(a) $P = ac$, $ZZ(P) = ac$, with locus w at string depth 2 (b) $P = bac$, $ZZ(P) = acb$, with locus w at string depth 3

Fig. 4: For each node v , the red square contains $\text{sd}(v)$ and the green rectangle $(\text{EV}(v), \text{OD}(v))$. The locus of $ZZ(P)$ is w .

$d_i = \text{sd}(u_i) - \text{sd}(v) - 1$ whenever u_i is a leaf. Let $\mathcal{L}(v)$ be the set of loci in the subtree rooted at v , excluding loci that end at a terminal symbol. For any locus $x = (u, \ell) \in \mathcal{L}(v)$, let $\delta_v(x) = \ell - \text{sd}(v)$. We then define

$$\text{EV}(v) = |\{x \in \mathcal{L}(v) : \delta_v(x) > 0 \text{ and } \delta_v(x) \equiv 0 \pmod{2}\}|,$$

and

$$\text{OD}(v) = |\{x \in \mathcal{L}(v) : \delta_v(x) > 1 \text{ and } \delta_v(x) \equiv 1 \pmod{2}\}|.$$

Thus, $\text{EV}(v)$ counts the loci at a positive even distance from v , while $\text{OD}(v)$ counts the loci at an odd distance greater than 1 from v ; the loci at distance one are covered by the $|\text{ch}(v)|$ term. We define the following recurrences:

$$\begin{aligned} \text{EV}(v) &= \sum_{i=1}^k \left(\left\lfloor \frac{d_i}{2} \right\rfloor + \begin{cases} \text{EV}(u_i) & \text{if } d_i \text{ is even,} \\ \text{OD}(u_i) + |\text{ch}(u_i)| & \text{if } d_i \text{ is odd,} \end{cases} \right) \\ \text{OD}(v) &= \sum_{i=1}^k \left(\left\lfloor \frac{d_i - 1}{2} \right\rfloor + \begin{cases} \text{OD}(u_i) + |\text{ch}(u_i)| & \text{if } d_i \text{ is even,} \\ \text{EV}(u_i) & \text{if } d_i \text{ is odd,} \end{cases} \right) \end{aligned}$$

For a leaf node v , we set $\text{EV}(v) = 0$ and $\text{OD}(v) = 0$, which serves as the base case of the recurrence.

We compute $\text{EV}(v)$ and $\text{OD}(v)$, for every v in $ZZT(T)$. This takes $\mathcal{O}(n)$ time via a single DFS traversal of $ZZT(T)$, provided that $ZZT(T)$ has been constructed.

Lemma 2. Let v be a node of $ZZT(T)$. Then the quantities $\text{EV}(v)$ and $\text{OD}(v)$, defined by the recurrences above, correctly compute the contribution of the subtree rooted at v to CC in the even and odd parity cases, respectively.

Proof. We prove the claim by induction on the size of the subtree rooted at v . If v is a leaf, then the subtree of v contains no outgoing edges, and hence contributes nothing. Therefore $\text{EV}(v) = \text{OD}(v) = 0$, as required. Now let v be an internal node, with children $u_1, \dots, u_k$, and let d_i denote the length of the edge label from v to u_i , as defined by the construction. Since the contribution of the subtree rooted at v is the sum of the contributions of the subtrees rooted at its children together with the contribution of the edges incident to v , it suffices to verify the recurrence for each child independently.

Consider first the even parity case. The contribution accumulated along the edge from v to u_i is $\lfloor d_i/2 \rfloor$. If d_i is even, then the parity state at u_i remains even, and the contribution

of the subtree rooted at u_i is $EV(u_i)$. If d_i is odd, then the parity state at u_i becomes odd; in this case, the contribution from the subtree rooted at u_i is $OD(u_i)$, together with the additional term corresponding to the children of u_i , namely $|ch(u_i)|$. This is exactly the recurrence defining $EV(v)$.

The argument for $OD(v)$ is analogous. When the parity state at v is odd, the contribution along the edge to u_i is $\lfloor (d_i - 1)/2 \rfloor$, and the parity state at u_i is determined by the parity of d_i in the same way as specified by the recurrence. The resulting contribution is therefore exactly $OD(v)$.

By induction, $EV(v)$ and $OD(v)$ correctly compute the contribution of the subtree rooted at v to CC . $\square$

Querying. Given a pattern P of length m , we first construct $ZZ(P)$ and locate its locus (v, m) in $ZZT(T)$ in $\mathcal{O}(m)$ time. Let $d = sd(v) - m$. By Lemma 2, the contribution of the subtree rooted at v is given by $EV(v)$ in the even case and by $OD(v)$ in the odd case. It remains to account for the contribution of the path segment between the locus and v , which is $\lfloor d/2 \rfloor$, together with the additional boundary contribution $|ch(v)|$ in the odd case. Therefore,

$$CC(P, T) = \begin{cases} \lfloor d/2 \rfloor + EV(v) & \text{if } d \text{ is even,} \\ \lfloor d/2 \rfloor + OD(v) + |ch(v)| & \text{if } d \text{ is odd.} \end{cases}$$

The query time is clearly $\mathcal{O}(m)$.

Example 11. Consider the $ZZT(T)$ in Fig. 4. We compute $EV(v)$ and $OD(v)$ for every node v via a DFS traversal.

For each leaf node $\ell_i, i \in [1, 4]$, we have $EV(\ell_i) = OD(\ell_i) = 0$. Node v_2 has two children: ℓ_1 with edge length $d_1 = 8 - 6 - 1 = 1$, and ℓ_2 with edge length $d_2 = 9 - 6 - 1 = 2$.

$$\begin{aligned} EV(v_2) &= (\lfloor 1/2 \rfloor + OD(\ell_1) + |ch(\ell_1)|) + (\lfloor 2/2 \rfloor + EV(\ell_2)) = 1 \\ OD(v_2) &= (\lfloor 0/2 \rfloor + EV(\ell_1)) + (\lfloor 1/2 \rfloor + OD(\ell_2) + |ch(\ell_2)|) = 0 \end{aligned}$$

Similarly, we compute $EV(v_3) = 2$ and $OD(v_3) = 0$, and $EV(v_1) = 5$ and $OD(v_1) = 5$; see Fig. 4. We now answer two CC queries: (1) $P = ac$ in Fig. 4a, where $|P|$ is even. We compute $ZZ(P) = ac$ and locate its locus $w = (v_1, 2)$ in $ZZT(T)$. Since $sd(v_1) = 4$, we have $d = sd(v_1) - |P| = 2$, which is even, so $CC(P, T) = \lfloor d/2 \rfloor + EV(v_1) = 1 + 5 = 6$. (2) $P = bac$ in Fig. 4b, where $|P|$ is odd. We compute $ZZ(P) = acb$ and locate its locus $w = (v_1, 3)$. Now $d = sd(v_1) - |P| = 1$, which is odd, so the remaining letter is combined with the first letter of every outgoing edge of v_1 , contributing $|ch(v_1)| = 2$ additional length-2 fragments, giving $CC(P, T) = \lfloor d/2 \rfloor + OD(v_1) + |ch(v_1)| = 0 + 5 + 2 = 7$.

Main Result. From the above, we directly obtain:

Theorem 4. CC can be solved using an index of $\mathcal{O}(n)$ size supporting $\mathcal{O}(m)$ -time queries. The index can be constructed in $\mathcal{O}(n)$ time and space given $ZZT(T)$.

VII. INDEXES FOR TCPR

A. Baseline Index for TCPR

We construct Navarro's SA-based index from [6] (see Section VIII), which, given P and q , reports, for every string pair (L, R) in $\mathcal{C}_T(P, q)$, the interval $[s, e]$, such that $\{SA[s], \dots, SA[e]\}$ are exactly the occurrences of LPR in T . Upon a TCPR query, we evaluate the scoring function $f_T(LPR)$ for each (L, R) pair and select the top- k candidates,

breaking ties arbitrarily. The construction takes $\mathcal{O}(n)$ time and space. The query time is $\mathcal{O}(m + |\mathcal{C}_T(P, q)| \cdot \mathcal{T}_{f_T})$, where $\mathcal{T}_{f_T}$ is the time required to evaluate f_T on LPR ; and the top- k selection can then be performed in linear time with the algorithm of [33]. Note that the LPR strings are not known in advance; precomputing their scores would therefore require enumerating all possible pairs (P, q) , and thus $\Theta(n^2)$ admissible LPR strings, which is prohibitively expensive.

B. Our Index for TCPR

Our index exploits ZZT , which compactly represents the distinct string pairs (L, R) for every P , and a geometric data structure to quickly retrieve the top- k of these pairs w.r.t. the scoring function f_T , which comprise the TCPR query answer.

Construction. Construct $ZZT(T)$ using Theorem 1. For each node w of $ZZT(T)$, compute the rank of w in a preorder traversal of $ZZT(T)$. This rank is called the *preorder rank* of w and denoted by $pre(w)$. The root has preorder rank 1. The preorder rank is used to locate pairs in the query answer. Then, for every node w in $ZZT(T)$, construct the following three-dimensional point

$$(pre(w), sd(w), sd(parent(w))),$$

and weigh it by $f_T(str(w))$, where f_T is the chosen scoring function; see subsection "Scoring Functions" below. The first dimension will ensure that every reported element is of the form LPR (i.e., the locus of LPR "descends" from the locus of P), and the second and third dimensions will ensure that the length constraints are satisfied (i.e., $|L| = |R| = q$ and $m = |P|$). Let S be the set of all such points. We preprocess S into the data structure described in Section II for top- k orthogonal range reporting, which can be constructed in $\tilde{\mathcal{O}}(n)$ time [25]. Thus, the total construction time is $\tilde{\mathcal{O}}(n)$.

Querying. Given a pattern P of length m , we construct $ZZ(P)$ and locate its locus (u, m) in $ZZT(T)$ in $\mathcal{O}(m)$ time. For the given $q \geq 0$, every descendant node w of u with

$$sd(parent(w)) < m + 2q \leq sd(w)$$

corresponds to a distinct element of $\mathcal{C}_T(P, q)$. Consider the locus of such an element. It corresponds to a string of the form LPR with $|L| = |R| = q$ and $m = |P|$. Accordingly, we report the top- k weighted points in $S \cap Q$, where

$$Q = [pre(u), pre(u')] \times [m + 2q, \infty) \times [0, m + 2q],$$

and u' is the rightmost leaf descendant of u . We now argue for correctness. By the preorder rank assignment, we have that $pre(w) \in [pre(u), pre(u')]$ if and only if w is a descendant of u ; and then the second and third dimensions of the query box guarantee that $|LPR|$ is exactly $m + 2q$ as desired. We have that the prefix of $str(w)$ of length $m + 2q$ is $ZZ(LPR)$. Let i be such that $ZZ(LPR)$ is a prefix of string Z_i . Then LPR occurs at position $(i - \lfloor \ell/2 \rfloor + 1)$ of T , with $\ell = m + 2q$.

The time for reporting the top- k points is $\tilde{\mathcal{O}}(k)$, and so the total query time becomes $\mathcal{O}(m) + \tilde{\mathcal{O}}(k)$.

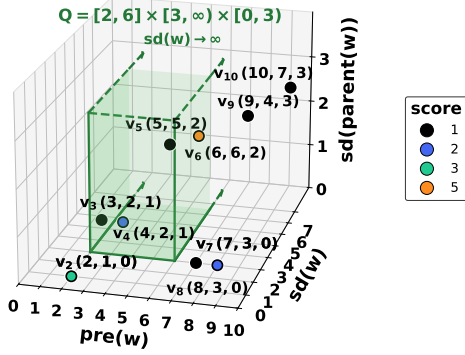


Fig. 5: Each point in S is colored based on its weight (score), assigned by f_T . The green hyperrectangle corresponds to Q .

Example 12. Let $T = \text{banana}$, the $\text{ZZT}(T)$ in Fig. 1(c), and a scoring function f_T . We perform a preorder traversal of $\text{ZZT}(T)$ and assign rank $\text{pre}(v_i) = i$ to each node $v_i, i \in [1, 10]$. For every node $w \in \text{ZZT}(T)$ except the root node v_1 , we construct a three-dimensional point $(\text{pre}(w), \text{sd}(w), \text{sd}(\text{parent}(w)))$, which is weighed by $f_T(\text{str}(w))$, as shown in Fig. 5. For example, for v_4 , we have $\text{pre}(v_4) = 4$, $\text{sd}(v_4) = |\text{str}(v_4)| = |an| = 2$, and $\text{sd}(\text{parent}(v_4)) = \text{sd}(v_2) = |a| = 1$. Thus, for v_4 , we construct a three-dimensional point $(4, 2, 1)$, which is assigned a weight $f_T(\text{str}(v_4)) = 2$. The set S is comprised of all such three-dimensional points constructed based on $v_2, \dots, v_{10}$, and then we construct the data structure from [25] on S . The points in it have coordinates in $[0, 10] \times [0, 7] \times [0, 3]$. Let $P = a$ be a query pattern of length $m = 1$, $q = 1$, and $k = 1$. We construct $\text{ZZ}(P) = a$ and locate its locus $(v_2, 1)$, which has $\text{pre}(v_2) = 2$ in $\text{ZZT}(T)$. As $m + 2q = 3$ and the rightmost leaf descendant of v_2 is v_6 , whose $\text{pre}(v_6) = 6$, we have

$$\begin{aligned} Q &= [\text{pre}(v_2), \text{pre}(v_6)] \times [m + 2q, \infty) \times [0, m + 2q] \\ &= [2, 6] \times [3, \infty) \times [0, 3]. \end{aligned}$$

Since $S \cap Q = \{(5, 5, 2), (6, 6, 2)\}$ and $k = 1$, we report the top-1 point, namely $(6, 6, 2)$. This point corresponds to v_6 , which is assigned a weight $f_T(\text{str}(v_6)) = 5$.

Main Result. From the above, we obtain:

Theorem 5. TCPR can be solved using an index of $\tilde{O}(n)$ size supporting $\mathcal{O}(m) + \tilde{O}(k)$ -time queries. The index can be constructed in $\tilde{O}(n)$ time and space given $\text{ZZT}(T)$.

Bounded-Length Optimization. Motivated by the fact that there is often an upper bound on the length of LPR in applications [8], we propose a parameterized version of our index that answers queries faster, has a much smaller size in practice, and can also be constructed faster and using less space. This index is parameterized by an integer $B \geq m + 2q$.

Let $T[1..n]$ be a string of length n over alphabet Σ . For every position $i \in [1, n]$, we define the B -bounded ZigZag string $Z_i(T)$ as the sequence $Z_i = T[i] T[i+1] T[i-1] T[i+2] T[i-2] \dots$, truncated after at most B letters; we drop (T) from $Z_i(T)$ when the context is clear. More precisely, we have $|Z_i| = \min(B, \max\{k \geq 1 : i + (-1)^k \lfloor \frac{k}{2} \rfloor \in [1, n]\})$.

Then, for all $k \in [1, |Z_i|]$, we have $Z_i[k] = T[i + (-1)^k \lfloor \frac{k}{2} \rfloor]$.

The counterpart of $\text{ZZT}(T)$ over the n B -bounded ZigZag strings of T can be constructed in $\mathcal{O}(n \log n)$ time using the same algorithm described in Section III. After the index construction, we can proceed with the construction of the TCPR index as described in the subsection “Construction” above. The query algorithm works as described in the subsection “Querying” above provided that $m + 2q \leq B$.

Scoring Functions. Recall from Section I that we can support any function $f_T : \Sigma^* \rightarrow \mathbb{R}_{\geq 0}$ computable in $\mathcal{O}(1)$ time after $\mathcal{O}(n)$ -time preprocessing. Our ZZT -based index relies on the fact that the inputs to f_T are precisely the LPR strings corresponding to explicit nodes in $\text{ZZT}(T)$. This permits using well-known importance measures such as *term frequency* (TF) [34], computed as $\text{occ}_T(LPR)$, and *span* (SP) [19], [20], computed as $|o_r - o_l|$, where o_r and o_l are the rightmost and leftmost occurrences of LPR in T , respectively. Other functions, such as *term proximity* (TP) [26], [35], [36], computed as $\min_{i \in [1, t]} (o_{i+1} - o_i)$ for occurrences $o_1, \dots, o_t$ of LPR in ascending order, can also be used as f_T , but their preprocessing cost must be accounted for in construction.

VIII. RELATED WORK

String indexing and pattern retrieval have been extensively studied, including work on full-text indexes and on top- k retrieval; see [34] for a textbook. Our work falls into the emerging area of contextual pattern matching.

As discussed in Section I, Navarro [6] introduced the problem of reporting the context of a given pattern, and Li et al. [8] introduced the problem of counting the context size of a pattern and of mining all patterns of a given length with a sufficiently large context. These works employ classic indexes, such as suffix or prefix trees [27]. There are three indexes that report all pairs in $\mathcal{C}_T(P, q)$ for a pattern P . The first index [6] uses a suffix array (SA) and other auxiliary data structures; it has $\mathcal{O}(|P| + |\mathcal{C}_T(P, q)|)$ query time and uses $\mathcal{O}(n)$ space. The second index [6] has $\mathcal{O}(|P| \log \log n + |\mathcal{C}_T(P, q)| \log n)$ query time and uses $\mathcal{O}(\bar{r} \log(n/\bar{r}))$ space, where $\bar{r}$ is the maximum number of equal-letter runs in the BWT (Burrows–Wheeler transform) of T and of its reverse. The third index [7] has $\mathcal{O}(|P| + |\mathcal{C}_T(P, q)| \log q \log(n/e))$ query time, where e is the number of equal-letter runs in the BWT of T , and uses $\mathcal{O}(e \log(n/e))$ space. The query time for the third index is slightly worse than that for the second. The last two indexes are of theoretical interest, as explained in [8]. Our work differs in that it introduces: (I) different contextual queries (which are not answered by the above indexes) and specialized indexes for them; and (II) ZZT , a general index for answering contextual queries efficiently, which allows directly retrieving all the contextual information for a pattern.

The family of affix trees [37], [38] and affix arrays [39] is closely related to ZZT : these indexes support bidirectional search, allowing a matched pattern to be extended both to the left and to the right, while maintaining its occurrences during the search. Similar functionality is provided by BWT-

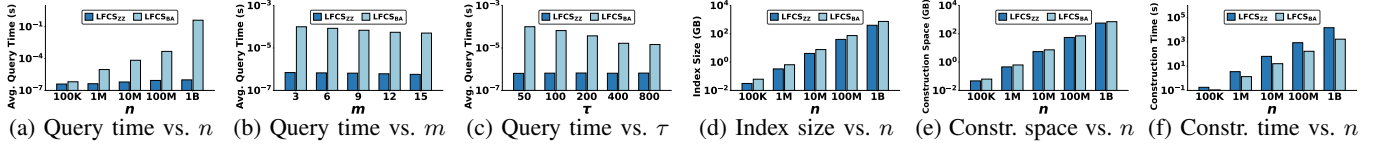


Fig. 6: Our LFCS index vs. LFCS_{BA} on SDSL.

TABLE V: Dataset characteristics.

Datasets	Domain	Length n	Alphabet size $ \Sigma $
WIKI [43]	biographical article	474,214,798	36
BST [44]	github repository	3,000,000,000	68
SDSL [45]	github repository	3,000,000,000	67
SARS [46]	bioinformatics	3,000,000,000	4
CHR [47]	bioinformatics	3,000,000,000	4

based bidirectional indexes [40], [41], [42]. However, all these indexes *are not alternatives* to ZTT for our purposes. They are designed to navigate from a known pattern to its bidirectional extensions, whereas our contextual queries require efficiently aggregating over all balanced contexts LPR , with $|L| = |R|$, around a given pattern P . In particular, these indexes do not by themselves provide the contextual information associated with P in the compacted form needed by ZTT.

IX. EXPERIMENTAL EVALUATION

Datasets. We used 5 publicly available, large-scale datasets of up to about 3 billion letters that were also used in related work [8]; see Table V. WIKI is a collection of Wikipedia articles [43]. BST and SDSL represent all commits from two GitHub repositories (Boost [44] and SDSL [45]), retrieved from [48]. SARS is a genomic dataset [46]. CHR is a dataset from the 1,000 Genomes Project [49] representing chromosome 19 sequences from 1,000 human haplotypes [47].

We report below results for one dataset per query type; the results for the other datasets are in Appendix.

Methods. Since *no* out-of-the-box solution exists for the type of queries we consider (see Section VIII), we compared our LFCS, LCCS, and CC indexes, referred to as LFCS_{ZT}, LCCS_{ZT}, and CC_{ZT}, to their corresponding baselines, LFCS_{BA}, LCCS_{BA}, and CC_{BA}.

We consider our TCPR index both without and with the bounded-length optimization, referred to as TCPR_{ZT} and TCPR_{ZT}, respectively. In both cases, we did not use the data structure from Section VII-B for top- k queries, since its $\mathcal{O}(n \log^d n)$ space usage [25] is prohibitive for our dataset sizes; here, we have $d = 3$. We instead used an R*-tree [50] to index the three-dimensional points for orthogonal range reporting (ORR). The R*-tree was shown to outperform other ORR data structures [8], such as Range trees [51], [52] and KD trees [53]. The R*-tree uses $\mathcal{O}(n)$ space; and the f_T scores were stored as satellite information. After querying the R*-tree for ORR, we obtain all points in $S \cap Q$ (see Section VII-B), and then select the top- k ones by their f_T scores.

We compared TCPR_{ZT} to the baseline from Section VII-A, referred to as TCPR_{BA}, using the TF, SP, and TP scoring functions from Section VII-B. TCPR_{ZT} was worse than TCPR_{ZT} in all efficiency measures and worse than TCPR_{BA} in all measures except query time; see Section IX-E.

Setup. We used all relevant measures of efficiency [54] (i.e., query time; index size; construction time; and construction space) in our evaluation, each for the problem parameter(s) that affect it. We measured query and construction time using the `chrono` C++ library, construction space using `/usr/bin/time -v`, and index size using `mallinfo2`.

Note that for the pattern length m , we used the same values as in [8], and $m = 9$ by default. When we varied n , we used increasingly longer prefixes of the dataset. As query patterns, we used the top-1000 most frequent length- m substrings (in the collection $\mathcal{C}$ for LCCS and in the text T for all other query types). In LFCS and LCCS, we used by default $\tau = 100$ and $\tau = 5$, respectively. In TCPR, by default we used $q = 9$ and varied it from 3 to 15, and $k = 10$ and varied it from 5 to 80. We set $B = m + 2q$, which gave the best performance across all measures; by default, $B = m + 2q = 27$. These values reflect realistic pattern and context sizes [8].

All experiments were conducted on an AMD EPYC 7282 CPU with 1T RAM and 1T disk space. All indexes were implemented in C++. *Our code and datasets are in [55].*

A. LFCS

Query time. Fig. 6a shows that, for varying n , our LFCS_{ZT} index offers *more than two orders of magnitude faster query time* on average (and up to five orders of magnitude faster) than LFCS_{BA}, and its query time is *unaffected* by n , as expected by its complexity. This experiment shows the key benefit of our index: its query time is independent of $\text{occ}_T(P)$. This term increases with n and it is very large (e.g., for $n = 10^9$, it is 127,560 on average over all queries), making LFCS_{BA} impractical for realistically long strings.

Fig. 6b shows that LFCS_{ZT} is 12 *times faster* than LFCS_{BA} on average over all values of m and its query time grows more slowly than linearly in m . LFCS_{BA} becomes faster as m increases, since longer patterns occur fewer times in T (i.e., the term $\text{occ}_T(P)$ in the complexity of LFCS_{BA} becomes smaller) and induce fewer right extension candidates.

Fig. 6c shows that LFCS_{ZT} is 8 *times faster* on average over all τ values. Its query time is unaffected by τ , unlike that of LFCS_{BA}. The query time of LFCS_{BA} increases when τ decreases, as the DFS it employs prunes a smaller fraction of the subtree of the locus of P in the suffix tree $\text{ST}(T)$.

Index size. Fig. 6d shows that our LFCS index is 48% *smaller* on average over all n values, and 46% *smaller* when $n = 10^9$. The reason LFCS_{BA} occupies more space is that it uses two suffix trees. The size of both indexes scales linearly with n , as expected by their complexities.

Construction space. Fig. 6e shows analogous results to those of Fig. 6d. For example, LFCS_{ZT} needs 20% *less space* to be constructed than LFCS_{BA} for $n = 10^9$. LFCS_{BA} needs more

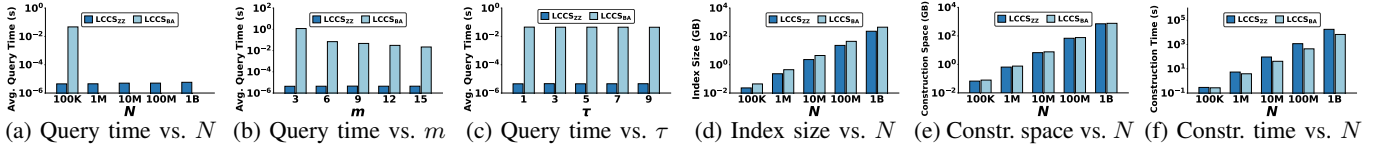


Fig. 7: Our LCCS index vs. LCCS_{BA} on SARS. In Fig. 7a, LCCS_{BA} did not terminate within 24 hours for $N \geq 10^6$.

space to be constructed due to its two suffix trees.

Construction time. Fig. 6f shows that LFC_{BA} is constructed 4 times faster than LFC_{ZT} on average over all n values. This is consistent with the complexities of these indexes. The construction time of our LFC_{BA} index was heavily influenced by that of ZTT(T) (e.g., constructing ZTT(T) took 85% of the total time for $n = 10^9$), which, however, is the key to its excellent query time performance.

B. LCCS

Query time. Fig. 7a shows that our LCCS_{ZT} index offers query answering that is *four orders of magnitude* faster than LCCS_{BA} and its query time is *unaffected* by N , as expected by its complexity. In fact, LCCS_{BA} did not terminate within 24 hours for $N \geq 10^6$, as the term $\text{occ}_T(P)$ in its complexity is very large (e.g., 2,323 on average over all queries). Thus, in the remaining of the section we used $N = 10^5$.

Fig. 7b shows that LCCS_{ZT} offers *four orders of magnitude faster query time* than LCCS_{BA} on average and its query time grows more slowly than linearly in m . LCCS_{BA} becomes faster as m increases, because the term $\text{occ}_T(P)$ in its complexity becomes smaller as m increases.

Fig. 7c shows that LCCS_{ZT} offers *four orders of magnitude faster query time* on average. Furthermore, its query time is unaffected by τ , unlike that of LCCS_{BA}. In this experiment, τ did not affect the query time of LCCS_{BA}, as the tested values were too small for DFS pruning to have a noticeable impact.

Index size. Fig. 7d shows that LCCS_{ZT} is 48% smaller than LCCS_{BA} on average over all N values, and 48% smaller when $N = 10^9$. The reason is the use of ZTT; the suffix trees in LCCS_{BA} occupy much more space. The size of both indexes scales linearly with N , as expected by their complexities.

Construction space. Fig. 7e shows analogous results to those of Fig. 7d. For example, LCCS_{ZT} needs 7% less space to be constructed than LCCS_{BA} for $N = 10^9$. The larger construction space of LCCS_{BA} is again due to its suffix trees.

Construction time. Fig. 7f shows that LCCS_{BA} is constructed two times faster than LCCS_{ZT} on average over all N values, as expected by the complexities of these indexes. Recall, however, that it is prohibitively expensive to query for realistically large string collections. The bottleneck for LCCS_{ZT} is the construction of ZTT.

C. CC

Query time. Fig. 8a shows that our CC_{ZT} index achieves query times that are *five orders of magnitude* faster than CC_{BA} and its query time is *unaffected* by n , as expected by its complexity. CC_{BA} did not terminate within 24 hours for $n \geq 10^6$, as the term $|\mathcal{L}(v)|$ in its query time bound is very large, which makes it impractical. For example, $|\mathcal{L}(v)|$ is $2 \cdot 10^7$ on average over all query patterns when $n = 10^6$. Thus, in the remainder of the section, we used $n = 10^5$.

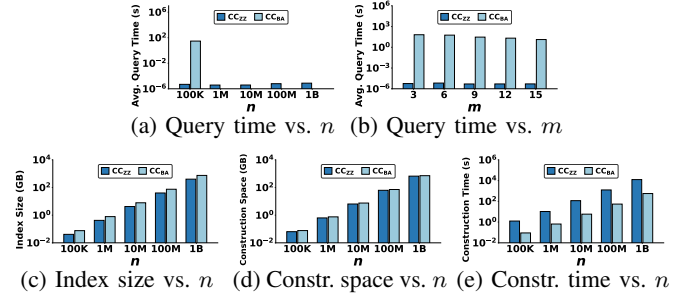


Fig. 8: Our CC index vs. CC_{BA} on BST. In Fig. 8a, CC_{BA} did not terminate within 24 hours for $n \geq 10^6$.

Fig. 8b shows that the query time for CC_{ZT} is *at least five orders of magnitude* faster than that of CC_{BA} and its query time grows more slowly than linearly in m . CC_{BA} gets faster as m increases, because the term $|\mathcal{L}(v)|$ decreases.

Index size. Fig. 8c shows that CC_{ZT} is 46% smaller on average over all n values and 46% smaller when $n = 10^9$, since ZTT occupies smaller space than the suffix trees used in CC_{BA}. The size of both CC_{ZT} and CC_{BA} scales linearly with n , as expected by their complexities.

Construction space. In Fig. 8d, we report analogous results to those of Fig. 8c, which are again due to the use of ZTT vs. the suffix trees. For example, CC_{ZT} needs 8% less space to be constructed than CC_{BA} for $n = 10^9$.

Construction time. Fig. 8e shows that CC_{BA} can be constructed faster than CC_{ZT}, by 19 times on average over all n values, as expected by their complexities. The bottleneck of CC_{ZT} is, as expected, the construction of ZTT. Recall, however, that the query time of CC_{BA} is prohibitive, and that, unlike query time, construction time is a one-off cost.

D. TCPR

Query time. Figs. 9a to 9c show that our TCPR_{ZT} index achieves query times that are *at least 6 and up to 44 times faster* than TCPR_{BA} across all n values and scoring functions. Since $|\mathcal{C}_T(P, q)|$ increases with n (e.g., from 652 to 10,967 on average over all patterns), TCPR_{BA} becomes substantially slower (e.g., 6 times slower as n increases from $2.5 \cdot 10^8$ to $3 \cdot 10^9$) and scales worse than TCPR_{ZT}. TCPR_{ZT} performs even better for the SP and TP scoring functions, as TCPR_{BA} evaluates them at query time, and these functions are more expensive to evaluate than TF; see Section VII-B.

Figs. 9d to 9f show that the query time of TCPR_{ZT} is *at least 4 and up to 60 times faster* than that of TCPR_{BA} as m varies. The query time for both indexes scales linearly with m , as expected by their complexities (here k is fixed). The query time of TCPR_{BA} is larger for SP and TP, as their computation time is larger compared to that of TF.

Figs. 10a to 10c show that for varying q the query time of TCPR_{ZT} is *at least 3 and up to 72 times faster* than that of

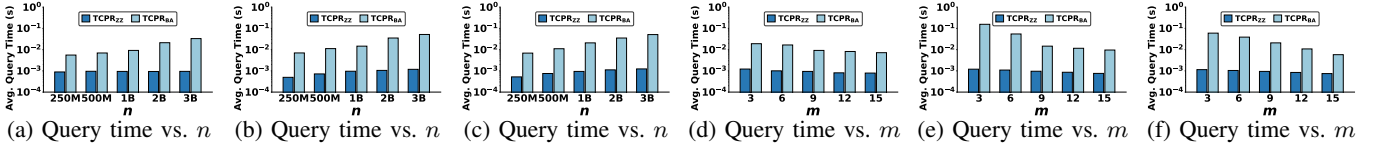


Fig. 9: Query time of our TCPR index vs. the baseline on CHR using the (a) TF, (b) SP, and (c) TP scoring function vs. n ; or using (d) TF, (e) SP, and (f) TP scoring function vs. m .

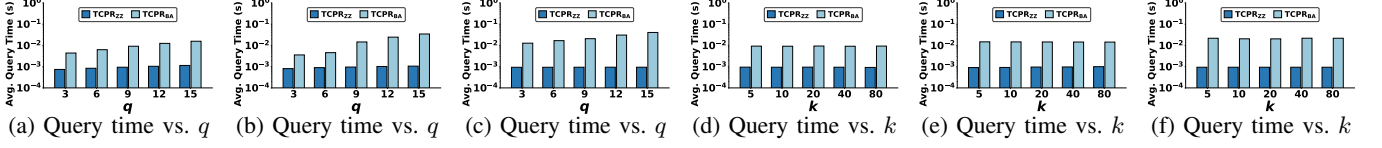


Fig. 10: Query time of our TCPR index vs. the baseline on CHR using the (a) TF, (b) SP, and (c) TP scoring function vs. q ; or using (d) TF, (e) SP, and (f) TP scoring function vs. k .

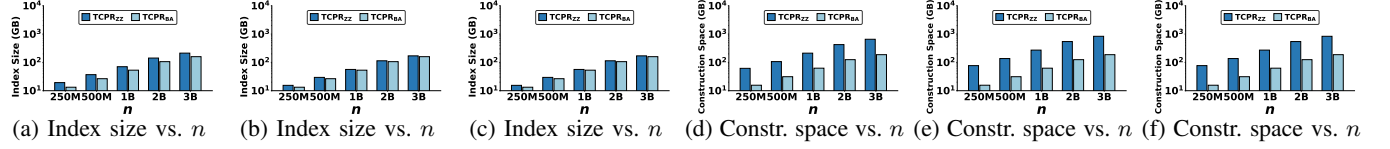


Fig. 11: Index size of our TCPR index vs. the baseline on CHR using the (a) TF, (b) SP, and (c) TP scoring function vs. n ; construction space of our TCPR index vs. the baseline on CHR using the (d) TF, (e) SP, and (f) TP scoring function vs. n .

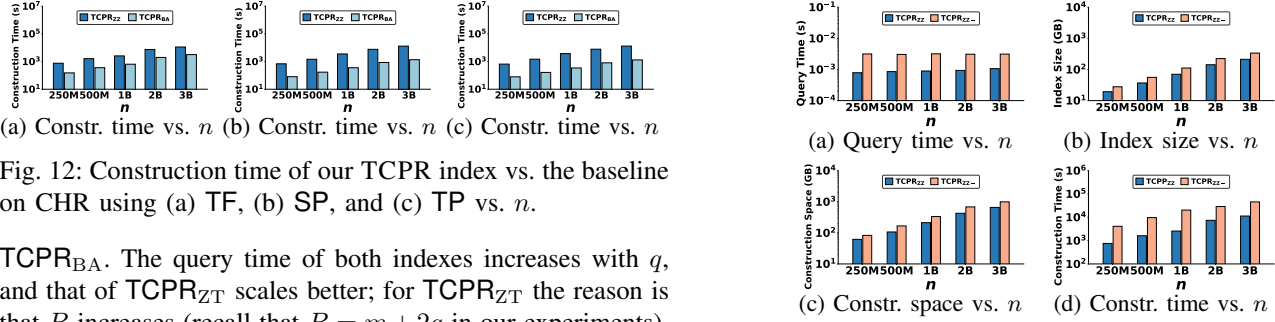


Fig. 12: Construction time of our TCPR index vs. the baseline on CHR using (a) TF, (b) SP, and (c) TP vs. n .

TCPR_{BA} . The query time of both indexes increases with q , and that of TCPR_{ZZ} scales better; for TCPR_{ZZ} the reason is that B increases (recall that $B = m + 2q$ in our experiments), while for TCPR_{BA} the reason is that $|\mathcal{C}_T(P, q)|$ increases.

Figs. 10d to 10f show that for varying k the query time of TCPR_{ZZ} is *at least 5 and up to 53 times faster* than TCPR_{BA} across all scoring functions. The query time of TCPR_{BA} is not affected substantially by k , as the only part that changes with k is the linear selection of pairs, which is very fast.

Index size. Figs. 11a to 11c show that the index size of both TCPR_{ZZ} and TCPR_{BA} scales linearly with n , as expected by the complexities, and that the size of the former is 16% larger.

Construction space. The results in Figs. 11d to 11f are analogous to those for index size in Figs. 11a to 11c. The gap between the two indexes is larger, as ZZT needs more space to be constructed.

Construction time. Fig. 12 shows that TCPR_{BA} can be constructed faster, but the gap from TCPR_{ZZ} becomes much smaller as n increases. Most of the construction time for TCPR_{ZZ} is spent in the ZZT construction. However, ZZT is also the key to the much faster query times of our index.

E. Ablation Study

We show that the bounded-length optimization is very effective: it substantially improved our TCPR index in all efficiency

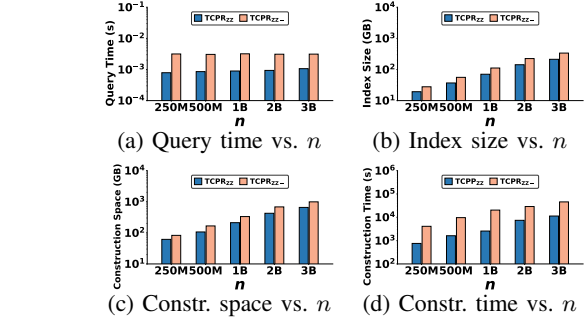


Fig. 13: (a) Query time, (b) index size, (c) construction space, and (d) construction time of TCPR_{ZZ} and $\text{TCPR}_{\text{ZZ-}}$ on CHR using the TF scoring function vs. n .

measures. We report results for the TF scoring function (those for SP and TP are analogous), and the default $B = 27$ in TCPR_{ZZ} . Fig. 13a shows that the query time of TCPR_{ZZ} is *at least 2.4 and up to 3.8 times faster* than that of $\text{TCPR}_{\text{ZZ-}}$. The index size, construction space, and construction time of TCPR_{ZZ} are smaller than those of $\text{TCPR}_{\text{ZZ-}}$ by 1.5, 1.6, and 3.9 times on average over all n values, respectively; see Figs. 13b, 13c, and 13d.

X. CONCLUSION

We introduced ZZT, a novel index that is useful for answering various types of contextual queries efficiently, and used it as basis to build specialized indexes for four practical contextual query types. Following [8], we considered context pairs with $|L| = |R|$, but our construction can be readily generalized to $|L| = g(|R|)$, for any $\mathcal{O}(1)$ -time computable g . The only change is in the definition of the ZigZag strings Z_i in Section III: each ZigZag move adds $g(|R|)$ letters instead of a single letter when moving left. For example,

when $g(|R|) = 2|R|$, we obtain $Z_i = T[i]T[i+1]T[i-1]T[i-2]T[i+2]T[i-3]T[i-4]\cdots$; after the first step $T[i+1]$ to the right, we add $2|R| = 2$ letters to the left, namely $T[i-1]T[i-2]$. Clearly, the ZigZag strings can then be sorted within the same time complexity, and the bounds for **ZZT** in Theorem 1 are identical. Another natural next step is to support dynamic texts. Since there is no practical full-text index for dynamic texts, designing dynamic versions of **ZZT** or our specialized indexes is a challenging problem left for future work. Since there is no practical full-text index for dynamic texts, designing dynamic versions of **ZZT** or our specialized indexes is a challenging problem left for future work.

REFERENCES

- [1] S. Goodwin, J. D. McPherson, and W. R. McCombie, “Coming of age: ten years of next-generation sequencing technologies,” *Nature Reviews Genetics*, vol. 17, pp. 333–351, 2016.
- [2] D. Jurafsky and J. H. Martin, *Speech and language processing: an introduction to natural language processing, computational linguistics, and speech recognition*, 2nd Edition, ser. Prentice Hall series in artificial intelligence. Prentice Hall, Pearson Education International, 2009.
- [3] P. A. Boncz, T. Neumann, and V. Leis, “FSST: fast random access string compression,” *Proc. VLDB Endow.*, vol. 13, no. 11, pp. 2649–2661, 2020.
- [4] E. Asselin, C. A. Melchor, and G. Jakllari, “Anomaly detection for web server log reduction: A simple yet efficient crawling based approach,” in *2016 IEEE Conference on Communications and Network Security, CNS 2016, Philadelphia, PA, USA, October 17-19, 2016*. IEEE, 2016, pp. 586–590.
- [5] M. Crochemore, C. Hancart, and T. Lecroq, *Algorithms on strings*. Cambridge University Press, 2007.
- [6] G. Navarro, “Contextual pattern matching,” in *String Processing and Information Retrieval - 27th International Symposium, SPIRE 2020, Orlando, FL, USA, October 13-15, 2020, Proceedings*, ser. Lecture Notes in Computer Science, C. Boucher and S. V. Thankachan, Eds. Springer, 2020, pp. 3–10.
- [7] P. Abedin, O. A. Chubet, D. Gibney, and S. V. Thankachan, “Contextual pattern matching in less space,” in *Data Compression Conference, DCC 2023, Snowbird, UT, USA, March 21-24, 2023*, A. Bilgin, M. W. Marcellin, J. Serra-Sagristà, and J. A. Storer, Eds. IEEE, 2023, pp. 160–167.
- [8] L. Li, D. Gibney, S. V. Thankachan, S. P. Pissis, and G. Loukides, “Contextual pattern mining and counting,” in *42nd IEEE International Conference on Data Engineering, ICDE 2026, Montréal, Canada, May 4-8, 2026*, 2026.
- [9] T. Chordia, R. Roll, and A. Subrahmanyam, “Order imbalance, liquidity, and market returns,” *Journal of Financial Economics*, vol. 65, no. 1, pp. 111–130, 2002.
- [10] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, “A survey on automated log analysis for reliability engineering,” *ACM Comput. Surv.*, vol. 54, no. 6, pp. 130:1–130:37, 2022.
- [11] A. Shawon, “Web log dataset,” 2019, online Judge (RUET OJ) server log dataset. [Online]. Available: <https://www.kaggle.com/datasets/shawon10/web-log-dataset>
- [12] N. Chinpattanakarn and C. Amornbunchornvej, “Framework for variable-lag motif following relation inference in time series using matrix profile analysis,” *ACM Trans. Knowl. Discov. Data*, vol. 19, no. 7, pp. 136:1–136:24, 2025.
- [13] L. Zhang, T. Jia, M. Jia, Y. Li, Y. Yang, and Z. Wu, “Multivariate log-based anomaly detection for distributed database,” in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*, R. Baeza-Yates and F. Bonchi, Eds. ACM, 2024, pp. 4256–4267.
- [14] E. Dąbrowska and D. Divjak, *Cognitive Linguistics: Key Topics*. Berlin, Boston: De Gruyter Mouton, 2019.
- [15] D. Kostić and W. Halfman, “Mapping explanatory language in neuroscience,” *Synthese*, vol. 202, no. 4, p. 112, 2023.
- [16] K. J. Loell, R. Z. Friedman, C. A. Myers, J. C. Corbo, B. A. Cohen, and M. A. White, “Transcription factor interactions explain the context-dependent activity of CRX binding sites,” *PLoS Comput. Biol.*, vol. 20, no. 1, 2024.
- [17] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap, “Compressive transformers for long-range sequence modelling,” *arXiv preprint arXiv:1911.05507*, 2019.
- [18] G. A. Miller, “WordNet: A lexical database for English,” *Commun. ACM*, vol. 38, no. 11, pp. 39–41, 1995.
- [19] T. Tao and C. Zhai, “An exploration of proximity measures in information retrieval,” in *SIGIR 2007: Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, Amsterdam, The Netherlands, July 23-27, 2007*, W. Kraaij, A. P. de Vries, C. L. A. Clarke, N. Fuhr, and N. Kando, Eds. ACM, 2007, pp. 295–302.
- [20] D. Hawking and P. B. Thistlewaite, “Proximity operators - so near and yet so far,” in *Proceedings of The Fourth Text REtrieval Conference, TREC 1995, Gaithersburg, Maryland, USA, November 1-3, 1995*, ser. NIST Special Publication, D. K. Harman, Ed. National Institute of Standards and Technology (NIST), 1995.
- [21] H. Chae, J. Park, S.-W. Lee, K. P. Nephew, and S. Kim, “Comparative analysis using K-mer and K-flank patterns provides evidence for CpG island sequence evolution in mammalian genomes,” *Nucleic Acids Research*, vol. 41, no. 9, pp. 4783–4791, 03 2013.
- [22] W. Matlock, S. Lipworth, B. Constantinides, T. E. Peto, A. S. Walker, D. Crook, S. Hopkins, L. P. Shaw, and N. Stoesser, “Flanker: a tool for comparative genomics of gene flanking regions,” *Microbial Genomics*, vol. 7, no. 9, p. 000634, 2021.
- [23] J. Li, T.-P. Chiu, and R. Rohs, “Predicting DNA structure using a deep learning method,” *Nature Communications*, vol. 15, no. 1, p. 1243, 2024.
- [24] L. C. K. Hui, “Color set size problem with application to string matching,” in *Combinatorial Pattern Matching, Third Annual Symposium, CPM 92, Tucson, Arizona, USA, April 29 - May 1, 1992, Proceedings*, ser. Lecture Notes in Computer Science, A. Apostolico, M. Crochemore, Z. Galil, and U. Manber, Eds., vol. 644. Springer, 1992, pp. 230–243.
- [25] S. Rahul, R. Gupta, R. Janardan, and K. S. Rajan, “Efficient top-k queries for orthogonal ranges,” in *WALCOM: Algorithms and Computation - 5th International Workshop, WALCOM 2011, New Delhi, India, February 18-20, 2011, Proceedings*, ser. Lecture Notes in Computer Science, N. Katoh and A. Kumar, Eds. Springer, 2011, pp. 110–121.
- [26] W. Hon, R. Shah, S. V. Thankachan, and J. S. Vitter, “Space-efficient frameworks for top-k string retrieval,” *J. ACM*, vol. 61, no. 2, pp. 9:1–9:36, 2014.
- [27] P. Weiner, “Linear pattern matching algorithms,” in *14th Annual Symposium on Switching and Automata Theory, Iowa City, Iowa, USA, October 15-17, 1973, 1973*, pp. 1–11.
- [28] M. Farach, “Optimal suffix tree construction with large alphabets,” in *38th Annual Symposium on Foundations of Computer Science, FOCS '97, Miami Beach, Florida, USA, October 19-22, 1997*. IEEE Computer Society, 1997, pp. 137–143.
- [29] U. Manber and E. W. Myers, “Suffix arrays: A new method for on-line string searches,” *SIAM J. Comput.*, vol. 22, no. 5, pp. 935–948, 1993.
- [30] D. Harel and R. E. Tarjan, “Fast algorithms for finding nearest common ancestors,” *SIAM J. Comput.*, vol. 13, no. 2, pp. 338–355, 1984.
- [31] T. Kasai, G. Lee, H. Arimura, S. Arikawa, and K. Park, “Linear-time longest-common-prefix computation in suffix arrays and its applications,” in *Combinatorial Pattern Matching, 12th Annual Symposium, CPM 2001 Jerusalem, Israel, July 1-4, 2001 Proceedings*, ser. Lecture Notes in Computer Science, A. Amir and G. M. Landau, Eds., vol. 2089. Springer, 2001, pp. 181–192.
- [32] M. Ruzic, “Constructing efficient dictionaries in close to sorting time,” in *Automata, Languages and Programming, 35th International Colloquium, ICALP 2008, Reykjavik, Iceland, July 7-11, 2008, Proceedings, Part I: Tack A: Algorithms, Automata, Complexity, and Games*, ser. Lecture Notes in Computer Science, L. Aceto, I. Damgård, L. A. Goldberg, M. M. Halldórsson, A. Ingólfssdóttir, and I. Walukiewicz, Eds. Springer, 2008, pp. 84–95.
- [33] M. Blum, R. W. Floyd, V. R. Pratt, R. L. Rivest, and R. E. Tarjan, “Time bounds for selection,” *J. Comput. Syst. Sci.*, vol. 7, no. 4, pp. 448–461, 1973.
- [34] R. Baeza-Yates and B. A. Ribeiro-Neto, *Modern Information Retrieval - the concepts and technology behind search, Second edition*. Pearson Education Ltd., Harlow, England, 2011.
- [35] Y. Rasolofo and J. Savoy, “Term proximity scoring for keyword-based retrieval systems,” in *Advances in Information Retrieval, 25th European Conference on IR Research, ECIR 2003, Pisa, Italy, April 14-16, 2003, Proceedings*, ser. Lecture Notes in Computer Science, F. Sebastiani, Ed. Springer, 2003, pp. 207–218.
- [36] R. Cummins and C. O’Riordan, “Learning in a pairwise term-term proximity framework for information retrieval,” in *Proceedings of the 32nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2009, Boston, MA, USA, July 19-23, 2009*, J. Allan, J. A. Aslam, M. Sanderson, C. Zhai, and J. Zobel, Eds. ACM, 2009, pp. 251–258.
- [37] M. G. Maaß, “Linear bidirectional on-line construction of affix trees,” *Algorithmica*, vol. 37, no. 1, pp. 43–74, 2003.
- [38] R. Cánovas and E. Rivals, “Full compressed affix tree representations,” in *2017 Data Compression Conference, DCC 2017, Snowbird, UT, USA, April 4-7, 2017*, A. Bilgin, M. W. Marcellin, J. Serra-Sagristà, and J. A. Storer, Eds. IEEE, 2017, pp. 102–111.

- [39] D. Strothmann, "The affix array data structure and its applications to RNA secondary structure analysis," *Theor. Comput. Sci.*, vol. 389, no. 1-2, pp. 278–294, 2007.
- [40] T. W. Lam, R. Li, A. Tam, S. C. K. Wong, E. Wu, and S. Yiu, "High throughput short read alignment via bi-directional BWT," in *2009 IEEE International Conference on Bioinformatics and Biomedicine, BIBM 2009, Washington, DC, USA, November 1-4, 2009, Proceedings*. IEEE Computer Society, 2009, pp. 31–36.
- [41] D. Belazzougui and F. Cunial, "Smaller fully-functional bidirectional BWT indexes," in *String Processing and Information Retrieval - 27th International Symposium, SPIRE 2020, Orlando, FL, USA, October 13-15, 2020, Proceedings*, ser. Lecture Notes in Computer Science, C. Boucher and S. V. Thankachan, Eds. Springer, 2020, pp. 42–59.
- [42] Y. Arakawa, G. Navarro, and K. Sadakane, "Bi-directional r-indexes," in *33rd Annual Symposium on Combinatorial Pattern Matching, CPM 2022, Prague, Czech Republic, June 27-29, 2022*, ser. LIPIcs, H. Bannai and J. Holub, Eds. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, pp. 11:1–11:14.
- [43] G. Navarro *et al.*, "Pizza & Chili corpus." [Online]. Available: <https://pizzachili.dcc.uchile.cl/repccorpus.html>
- [44] "Boost c++ libraries." [Online]. Available: <https://github.com/boostorg/boost>
- [45] S. Gog *et al.*, "SDSL Github repository." [Online]. Available: <https://github.com/simongog/sdsl-lite>
- [46] "National center for biotechnology information," 2024. [Online]. Available: <https://www.ncbi.nlm.nih.gov/datasets/genome/>
- [47] "Pangenome dataset," 2021. [Online]. Available: <https://github.com/koepl/phonix>
- [48] N. Prezza, "Get git revisions script." [Online]. Available: <https://github.com/nicolaprezza/get-git-revisions>
- [49] "1000 genomes project," 2008. [Online]. Available: <https://www.internationalgenome.org/>
- [50] N. Beckmann, H. Kriegel, R. Schneider, and B. Seeger, "The r*-tree: An efficient and robust access method for points and rectangles," in *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data, Atlantic City, NJ, USA, May 23-25, 1990*, H. Garcia-Molina and H. V. Jagadish, Eds. ACM Press, 1990, pp. 322–331.
- [51] J. L. Bentley, "Decomposable searching problems," *Inf. Process. Lett.*, vol. 8, no. 5, pp. 244–251, 1979.
- [52] G. S. Lueker, "A data structure for orthogonal range queries," in *19th Annual Symposium on Foundations of Computer Science, Ann Arbor, Michigan, USA, 16-18 October 1978*. IEEE Computer Society, 1978, pp. 28–34.
- [53] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Commun. ACM*, vol. 18, no. 9, pp. 509–517, 1975.
- [54] L. A. K. Ayad, G. Loukides, and S. P. Pissis, "Text indexing for long patterns: Anchors are all you need," *Proc. VLDB Endow.*, vol. 16, no. 9, pp. 2117–2131, 2023.
- [55] "Code and datasets for the paper: "ZigZag Trie: A Novel Index for Contextual Queries"." [Online]. Available: <https://github.com/LINGLI97/ZZT.git>

APPENDIX

For each of our four indexes, Section IX reported results on one representative dataset. We report here the same measurements on the remaining four datasets. The behavior of every index, and hence all conclusions drawn in Section IX, are analogous.

Figs. 14 and 15 show the query time, index size, construction space, and construction time of LFCS_{ZT} vs. LFCS_{BA} on BST, CHR, SARS, and WIKI. They are analogous to Figs. 6a–6f, which report the same measurements on SDSL in Section IX-A.

Figs. 16 and 17 show the same measurements for LCCS_{ZT} vs. LCCS_{BA} on BST, CHR, SDSL, and WIKI, and are analogous to Figs. 7a–7f, which report them on SARS in Section IX-B.

Figs. 18 and 19 show the same measurements for CC_{ZT} vs. CC_{BA} on CHR, SARS, SDSL, and WIKI, and are analogous to Figs. 8a–8e, which report them on BST in Section IX-C.

Figs. 20–25 show the same measurements for TCPR_{ZT} vs. TCPR_{BA} , using the TF, SP, and TP scoring functions, on BST, SARS, SDSL, and WIKI. They are analogous to Figs. 9–12, which report them on CHR in Section IX-D.

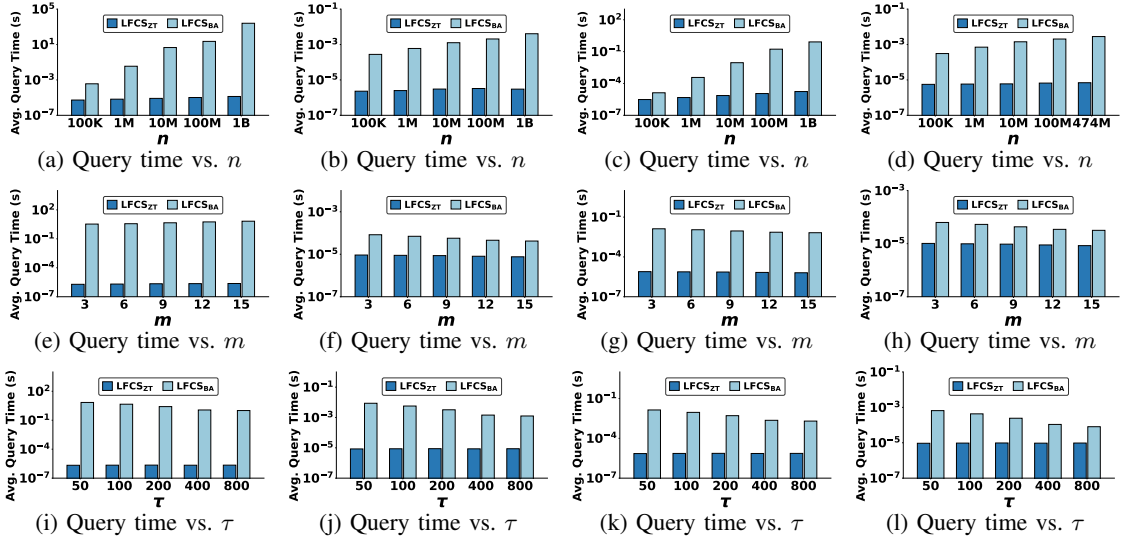


Fig. 14: Query time of our LFCS index vs. LFCS_{BA} on (a) BST, (b) CHR, (c) SARS, and (d) WIKI vs. n ; on (e) BST, (f) CHR, (g) SARS, and (h) WIKI vs. m ; on (i) BST, (j) CHR, (k) SARS, and (l) WIKI vs. τ .

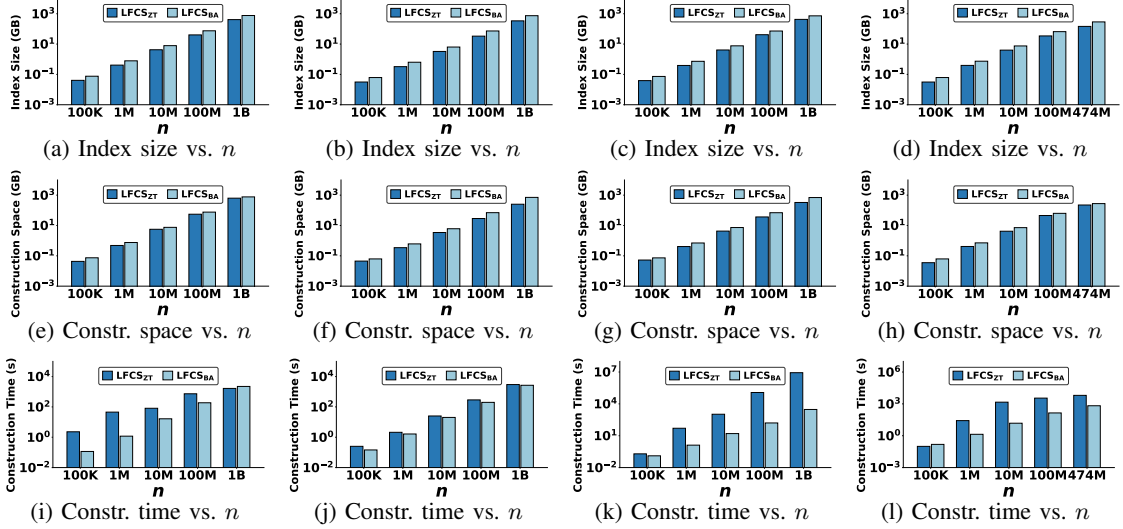


Fig. 15: Index size of our LFCS index vs. LFCS_{BA} on (a) BST, (b) CHR, (c) SARS, and (d) WIKI vs. n ; construction space of our LFCS index vs. LFCS_{BA} on (e) BST, (f) CHR, (g) SARS, and (h) WIKI vs. n ; construction time of our LFCS index vs. LFCS_{BA} on (i) BST, (j) CHR, (k) SARS, and (l) WIKI vs. n .

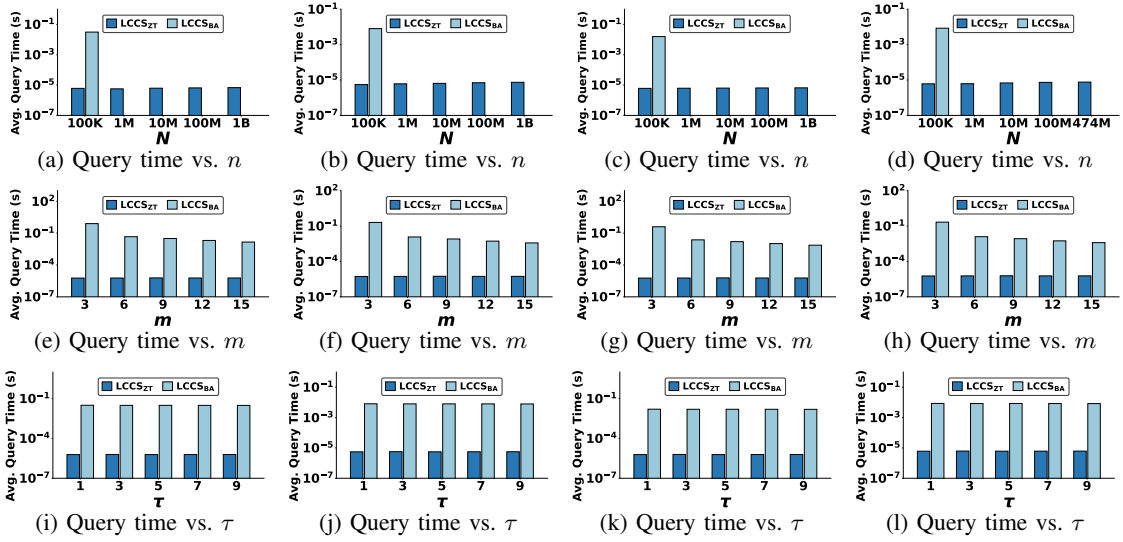


Fig. 16: Query time of our LCCS index vs. $LCCS_{BA}$ on (a) BST, (b) CHR, (c) SDSL, and (d) WIKI vs. n ; on (e) BST, (f) CHR, (g) SDSL, and (h) WIKI vs. m ; on (i) BST, (j) CHR, (k) SDSL, and (l) WIKI vs. τ .

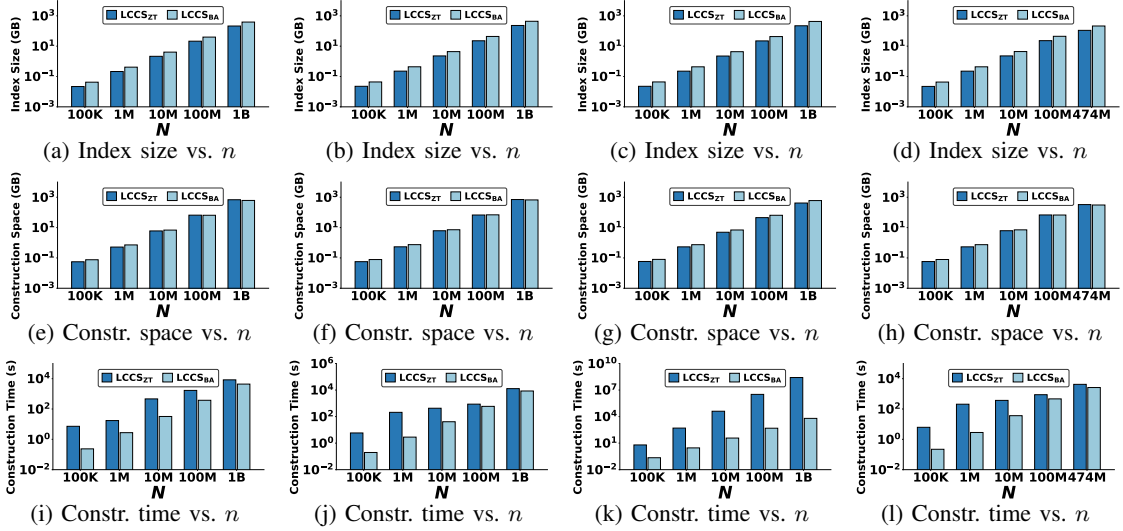


Fig. 17: Index size of our LCCS index vs. $LCCS_{BA}$ on (a) BST, (b) CHR, (c) SDSL, and (d) WIKI vs. n ; construction space of our LCCS index vs. $LCCS_{BA}$ on (e) BST, (f) CHR, (g) SDSL, and (h) WIKI vs. n ; construction time of our LCCS index vs. $LCCS_{BA}$ on (i) BST, (j) CHR, (k) SDSL, and (l) WIKI vs. n .

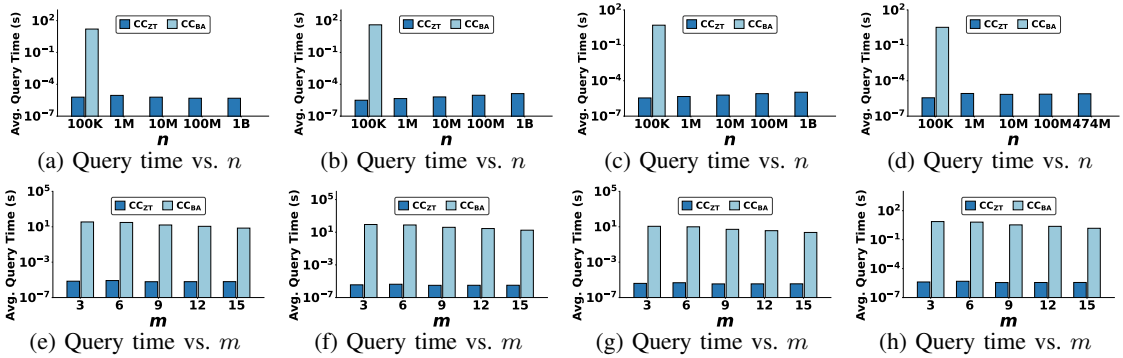


Fig. 18: Query time of our CC index vs. CC_{BA} on (a) CHR, (b) SARS, (c) SDSL, and (d) WIKI vs. n ; on (e) CHR, (f) SARS, (g) SDSL, and (h) WIKI vs. m .

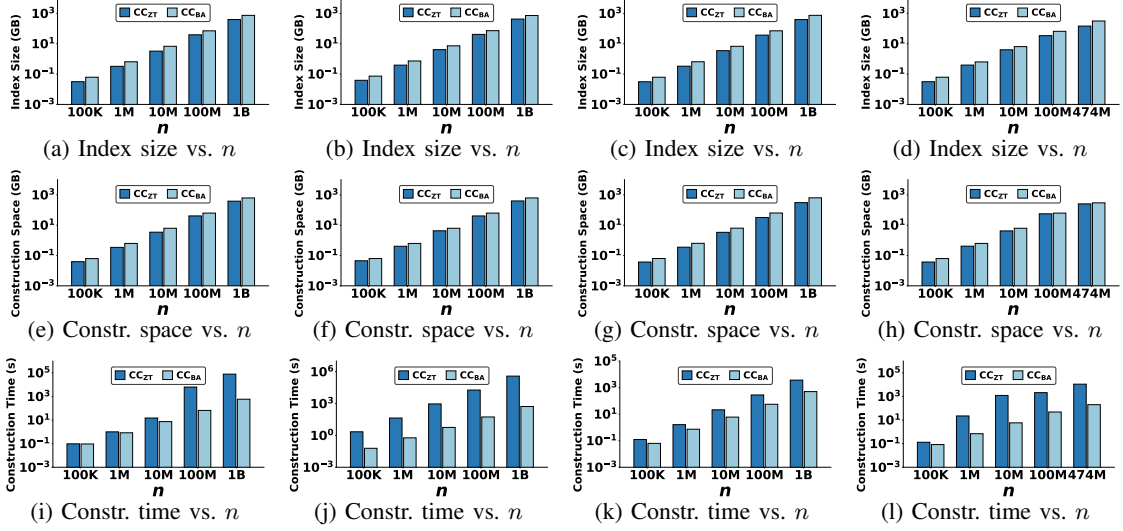


Fig. 19: Index size of our CC index vs. CC_{BA} on (a) CHR, (b) SARS, (c) SDSL, and (d) WIKI vs. n ; construction space of our CC index vs. CC_{BA} on (e) CHR, (f) SARS, (g) SDSL, and (h) WIKI vs. n ; construction time of our CC index vs. CC_{BA} on (i) CHR, (j) SARS, (k) SDSL, and (l) WIKI vs. n .

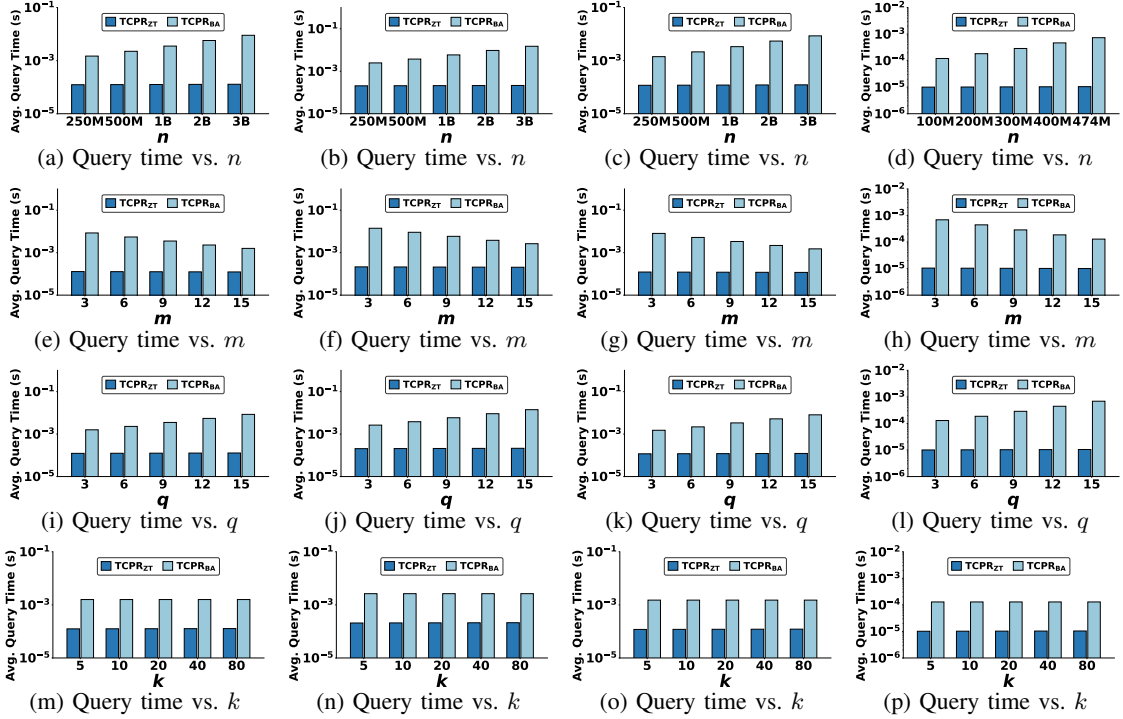


Fig. 20: Query time of our TCPR index with the TF scoring function vs. TCPR_{BA} on (a) BST, (b) SARS, (c) SDSL, and (d) WIKI vs. n ; on (e) BST, (f) SARS, (g) SDSL, and (h) WIKI vs. m ; on (i) BST, (j) SARS, (k) SDSL, and (l) WIKI vs. q ; on (m) BST, (n) SARS, (o) SDSL, and (p) WIKI vs. k .

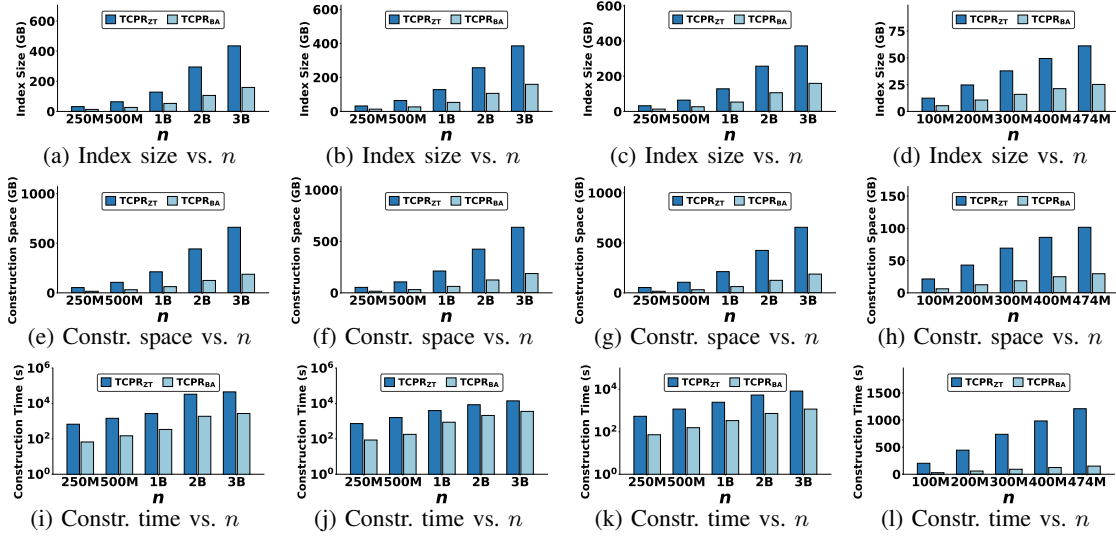


Fig. 21: Index size of our TCPR index with the TF scoring function vs. TCPR_{BA} on (a) BST, (b) SARS, (c) SDSL, and (d) WIKI vs. n ; construction space of our TCPR index with the TF scoring function vs. TCPR_{BA} on (e) BST, (f) SARS, (g) SDSL, and (h) WIKI vs. n ; construction time of our TCPR index with the TF scoring function vs. TCPR_{BA} on (i) BST, (j) SARS, (k) SDSL, and (l) WIKI vs. n .

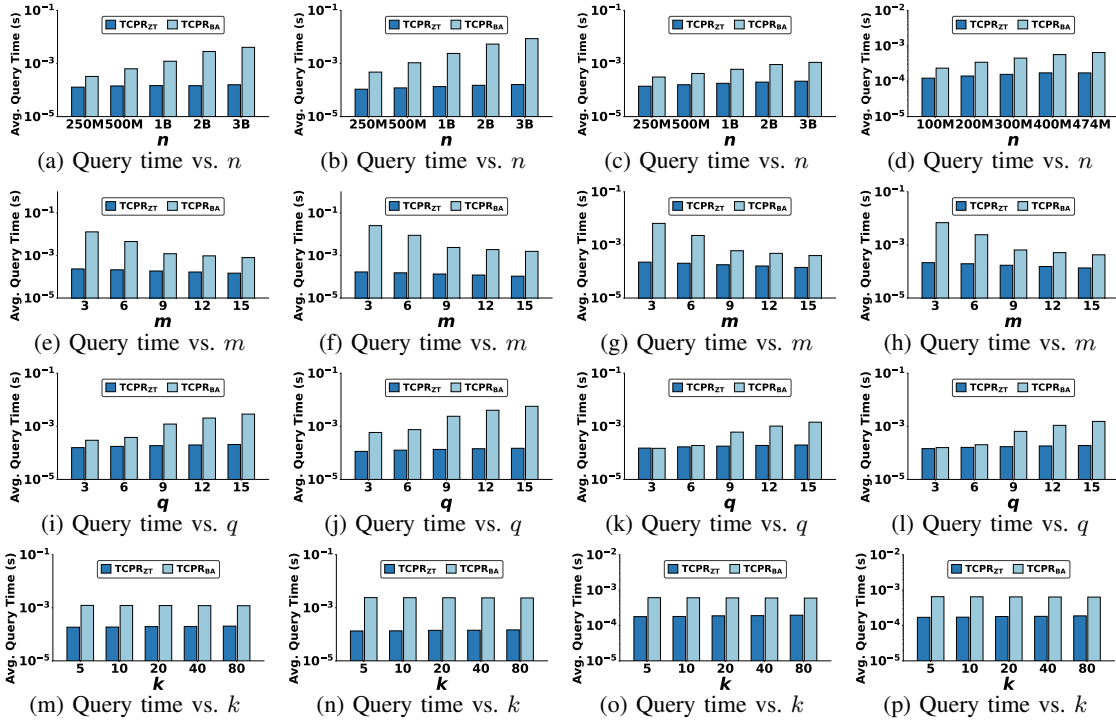


Fig. 22: Query time of our TCPR index with the SP scoring function vs. TCPR_{BA} on (a) BST, (b) SARS, (c) SDSL, and (d) WIKI vs. n ; on (e) BST, (f) SARS, (g) SDSL, and (h) WIKI vs. m ; on (i) BST, (j) SARS, (k) SDSL, and (l) WIKI vs. q ; on (m) BST, (n) SARS, (o) SDSL, and (p) WIKI vs. k .

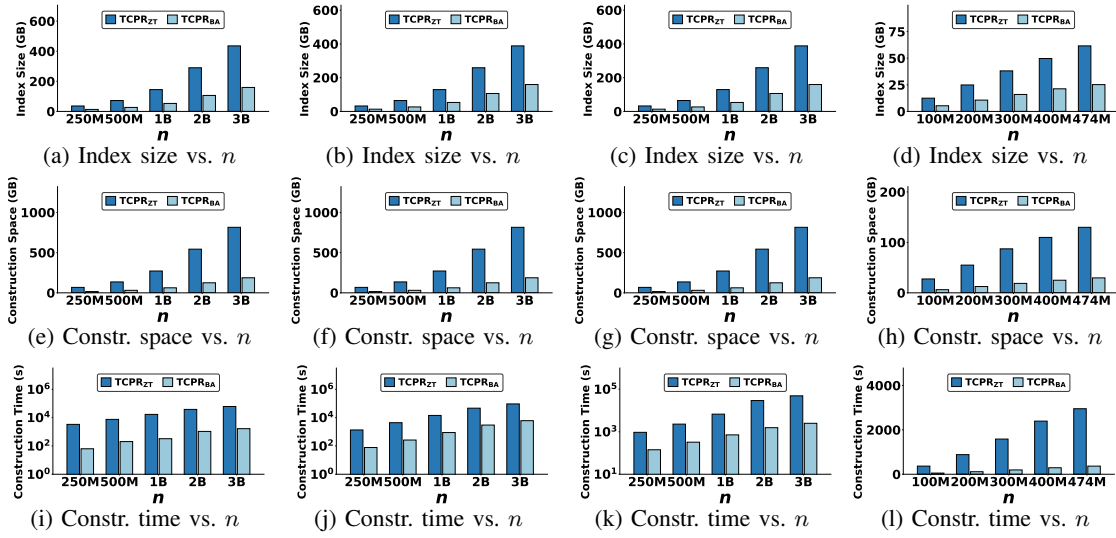


Fig. 23: Index size of our TCPR index with the SP scoring function vs. TCPR_{BA} on (a) BST, (b) SARS, (c) SDSL, and (d) WIKI vs. n ; construction space of our TCPR index with the SP scoring function vs. TCPR_{BA} on (e) BST, (f) SARS, (g) SDSL, and (h) WIKI vs. n ; construction time of our TCPR index with the SP scoring function vs. TCPR_{BA} on (i) BST, (j) SARS, (k) SDSL, and (l) WIKI vs. n .

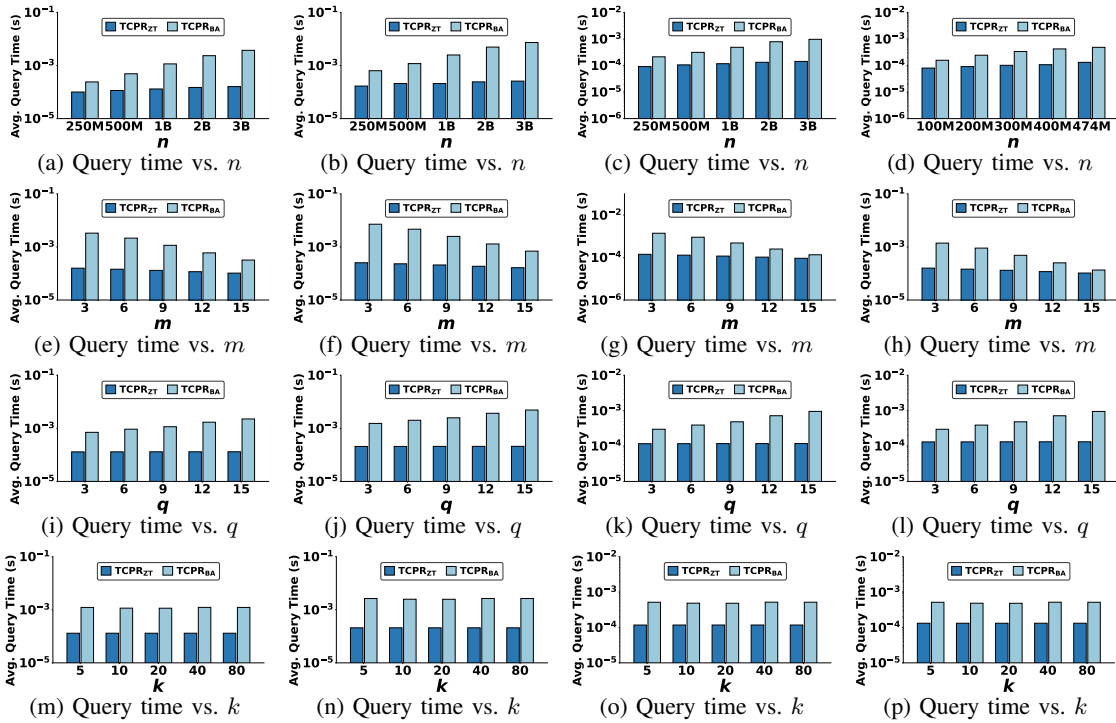


Fig. 24: Query time of our TCPR index with the TP scoring function vs. TCPR_{BA} on (a) BST, (b) SARS, (c) SDSL, and (d) WIKI vs. n ; on (e) BST, (f) SARS, (g) SDSL, and (h) WIKI vs. m ; on (i) BST, (j) SARS, (k) SDSL, and (l) WIKI vs. q ; on (m) BST, (n) SARS, (o) SDSL, and (p) WIKI vs. k .

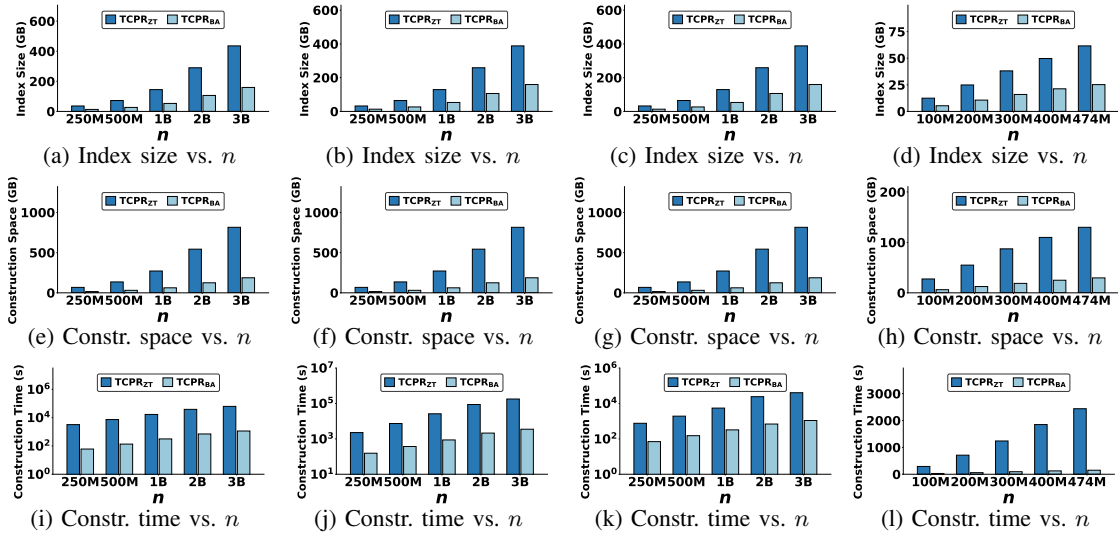


Fig. 25: Index size of our TCPR index with the TP scoring function vs. TCPR_{BA} on (a) BST, (b) SARS, (c) SDSL, and (d) WIKI vs. n ; construction space of our TCPR index with the TP scoring function vs. TCPR_{BA} on (e) BST, (f) SARS, (g) SDSL, and (h) WIKI vs. n ; construction time of our TCPR index with the TP scoring function vs. TCPR_{BA} on (i) BST, (j) SARS, (k) SDSL, and (l) WIKI vs. n .